HDF5 Audit Report

[NN]

Very abstract...

hello

Acknowledgments: This material is based upon work supported by the U.S. National Science Foundation under Federal Award No. 2534078. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.

Revision History

Version Number	Date	Comments
v1	July 4, 2026	Initial draft released for internal review.
v2	August 1, 2026	Public release.

Contents

1. Introduction

1.1. Purpose

1.2. Scope

Out of scope:

- PHDF5
- HDFView
- HSDS
- AI

1.2.1. Safety

1.2.2. Security

1.2.3. Privacy

The National Institute of Standards and Technology (NIST) defines privacy as "a condition that safeguards human autonomy and dignity through various means, including confidentiality, predictability, manageability, and disassociability." Historically, privacy was interpreted as preventing unauthorized access to or disclosure of Personally Identifiable Information (PII). However, NIST emphasizes that contemporary privacy risks extend far beyond traditional notions of confidentiality. Modern privacy concerns increasingly arise from inference, aggregation, behavioral profiling, secondary use of data, context changes, and surveillance effects [**nist_privacy_framework_2020**]. NIST emphasizes that privacy risks are distinct from cybersecurity risks and may emerge even when systems are secure, lawful, and functioning exactly as intended [**brooks2017privacy**]. HDF5 inherits many of the same classes of privacy risks identified by NIST, although these risks manifest differently in scientific and engineering environments.

HDF5 was designed to maximize data shareability, extensibility, portability, and long-term usability, and doesn't enforce data privacy or provide data protection mechanism. Its rich metadata model, self-describing structure, plugin architecture, and interoperability layers can expose information extending beyond the raw data stored within the file. Sensitive information in HDF5 ecosystems may emerge through HDF5 internal and user-defined metadata, for example, HDF5 attributes, object names, file organization, and objects properties. It may be exposed during HDF5 data lifecycle through provenance records, workflow artifacts, and correlations across multiple datasets and files.

A common misconception in scientific computing is that privacy concerns are resolved once data access is restricted or datasets are anonymized. However, privacy risks often persist even when traditional security mechanisms are correctly implemented. Restricting access does not prevent inference from metadata. Anonymization may remove direct identifiers while leaving contextual information sufficient for re-identification. Similarly, data reduction or aggregation does not eliminate the possibility that sensitive information may be reconstructed through correlations across files, workflows, or external repositories. In modern scientific ecosystems, privacy exposure frequently results from accumulation of data and metadata over time rather than disclosure of a single sensitive piece of raw data or metadata.

The privacy risks are amplified by current scientific practice of data openness and shareability. HDF5 datasets increasingly move across organization, systems, cloud platforms, workflows, and public and private repositories. Data are routinely repurposed beyond their original intent and analyzed by users who were not involved in their creation. Machine learning systems further amplify these concerns by discovering hidden relationships across data stored in HDF5 and enabling new forms of inference that may not have been anticipated when data were originally collected. As a result, data stored in the HDF5 files may experience evolving privacy characteristics through its lifecycle.

Therefore, privacy exposure in HDF5 is not a matter of unauthorized access, data breaches, or unsafe usage or malicious behavior, i.e., it is different from data security and safety. By design HDF5 file format and library do not guarantee

data privacy. In addition, privacy risks may emerge during legitimate scientific workflows, through data sharing, reuse, and analysis. In this document we audit privacy assets and different privacy exposure mechanisms (or surfaces) in HDF5 with the goal to raise awareness about privacy risks when using HDF5.

2. Motivation

2.1. Problem Statement

2.2. Goals

- Emphasize broader applicability for comparable OSEs

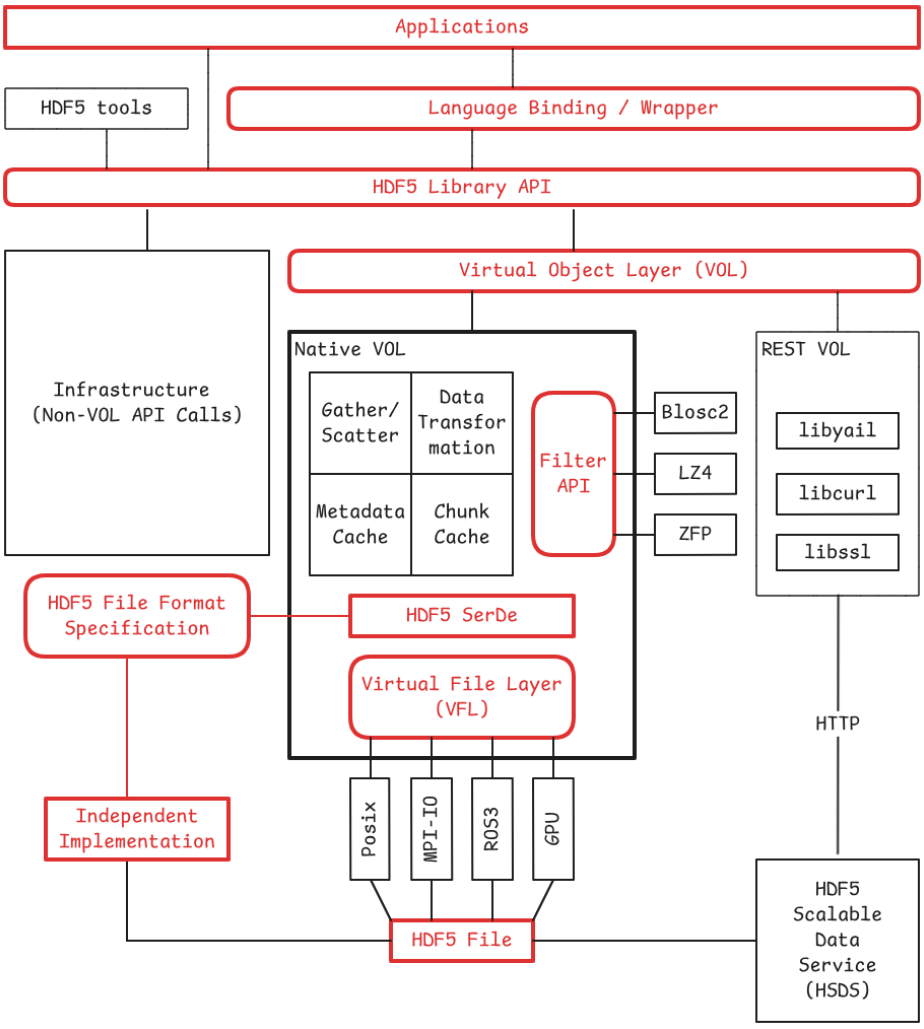

Figure 1 – HDF5 vulnerability sites (interfaces: rounded rectangles)

3. HDF5 Core Library and File Format

3.1. Accidents Waiting to Happen – HDF5 Safety

- Disk full [**forum-11714**]
- File corruption [**forum-file-corruption**]
- Long-term data accessibility and interpretability, see section 9

3.2. Verify and Verify – HDF5 Security

- Integer arithmetic
- File format parsing issues
- Malicious plugins, see section 4

A CVE history summary is included in Appendix A. The list mixes true HDF5-library issues, removed-tool issues, duplicate/rejected/not-HDF5 entries, and entries with no commit URL. The main pattern is not "many unrelated bugs"; it is repeated failure to validate file-derived structure sizes, offsets, counts, message lengths, continuation chunks, datatype metadata, or filter parameters before using them in low-level C memory operations. The 2025 entries alone show repeated heap overflows, use-after-free, null dereference, memory leak, and resource-consumption defects across object headers, file-space metadata, type conversion, filters, and metadata cache code.

The fix commentary confirms that several CVEs were symptoms of corrupted metadata not being rejected early. (Examples: the file-space page-size fix added a sanity check immediately after reading `page_size` from the file; the object-header-continuation fix rejects zero continuation chunk sizes; the self-referential continuation-message fix checks expected versus actual deserialized object-header chunks; and the cache-image fix added buffer-size arguments, overflow checks, input validation, and cleanup.)

The major root-cause categories are tighter than the CVE count suggests, and are summarized in Table 1.

Table 1 – Major root-cause categories in the HDF5 CVE history

Root-cause category	What it means for HDF5
unchecked file-derived sizes / counts / offsets	The dominant class. HDF5 metadata controls object layout, heap blocks, chunks, dataspaces, filters, links, and type descriptors. When those fields are malformed, C code later trusts them.
inconsistent internal bookkeeping	Attribute tables, cache images, continuation chunks, and free lists often needed separate "allocated capacity" versus "actual count" tracking.
decode/encode asymmetry	Several issues happen because decode did not reject bad messages, then encode/flush later assumed valid native structures.
arithmetic precondition failures	Divide-by-zero and multiplication-overflow checks were missing in chunking, layout, b-tree, filter, and dataspace paths.
unsafe cleanup/lifetime paths	Use-after-free, double-free, leaks, and null dereferences show that malformed input often breaks unwinding logic as much as the primary decode logic.
unbounded traversal, recursion, loop-progress, and resource-limit failures	External-reference/path-opening behavior is a policy boundary, not necessarily a parser memory-safety defect.
legacy tool attack surface	`gif2h5`, HDF to GIF conversion, `h5dump`, and `h5repack` appear repeatedly. Removal of `gif2h5` was a structural mitigation, not just a patch.
triage quality, duplicate/variant clustering, reachability, and fix-completeness metadata	Several entries have `commit_url:null`, "not filed against HDF5," "cannot reproduce," "not applicable," or tentative confidence. Those should not be counted as reviewed HDF5 fixes without qualification.

3.3. HDF5 Privacy: Assets and Exposure

3.3.1. Introduction

Privacy risks in the HDF5 core library and file format are not accidental; as discussed in the Introduction, they are systematic and arise from multiple factors. The design and implementation of the HDF5 file format and library present one important source of such risks. Additional risks emerge from scientific

workflows, software ecosystems, interoperability mechanisms, and evolving data usage contexts.

Before describing the assets that require protection in HDF5 and the mechanisms through which privacy exposure may occur, it is important to emphasize several fundamental characteristics of HDF5 and the audit approach adopted in this document.

- *HDF5 privacy primarily concerns accidental or unintended exposure of sensitive information rather than malicious attacks*. Malicious attacks and unauthorized access are primarily addressed through cybersecurity mechanisms and are outside the scope of this section.
- *HDF5, by design, does not provide privacy protections*. When privacy is a concern, users and organizations are responsible for protecting data throughout the entire data lifecycle. In this document, the term data refers not only to individual HDF5 objects within a single file, but also to collections of HDF5 files and external data accessible through the HDF5 library, connectors, and related software components.
- *HDF5 neither implements nor enforces privacy standards*. Furthermore, even when privacy standards and technical safeguards are in place, sensitive information may still be exposed through legitimate data processing, contextual interpretation, pattern discovery, inference, aggregation, and other analytical activities.
- *HDF5 does not manage authentication, authorization, or filesystem protection*. However, privacy risks extend beyond traditional security concerns. Sensitive information may be exposed during legitimate data use by authorized users, even within highly secure computing environments.
- *The risk of sensitive information exposure generally increases over the data lifecycle*. Sensitive information stored in HDF5 may be exposed during any stage of the lifecycle, including data creation, processing, storage, sharing, transfer, reuse, and archival.
- *HDF5 extensibility mechanisms and associated software ecosystems may alter the privacy characteristics of data*. Features such as filters, Virtual

Object Layer (VOL) connectors, Virtual File Drivers (VFDs), and external applications, including conversion and analysis tools, may introduce new privacy exposure surfaces or modify existing ones.

The remainder of this section focuses on the assets that require protection in HDF5, the mechanisms through which sensitive information may be exposed, and potential approaches for mitigating privacy risks.

3.3.2. HDF5 Assets and their Exposure

The primary assets requiring protection are the scientific data themselves. In HDF5, these include data stored in HDF5 objects, such as files, groups, datasets, datatypes, and dataspaces, which represent experimental observations, measurements, simulations, images, sensor outputs, and computational results.

Scientific data may be exposed through direct access using HDF5-based software, including command-line utilities, graphical viewers, and programming interfaces such as Python. However, privacy-sensitive information may also be inferred through examination of the low-level storage organization and auxiliary metadata embedded within the file to support portability, interoperability, and efficient data access. For example, string attributes visible through standard operating system utilities (e.g., the Unix strings command) may disclose dataset names, experimental identifiers, or other descriptive information. Similarly, inspection of dataset headers may reveal storage layout, chunk organization, compression methods, or data acquisition patterns (e.g., frame-based acquisition stored as chunked datasets). Examination of embedded strings may also disclose the use of external storage or references to datasets located in other HDF5 files.

It is important to distinguish between protecting access to data values and protecting information about data representation. Even when raw data are protected through mechanisms such as compression or encryption, information describing their storage organization, layout, or access methods may remain accessible and reveal sensitive contextual information about the underlying data or the processes that generated them.

Data Assets The primary assets requiring protection are the scientific data themselves. In HDF5, these include data stored in HDF5 objects, such as files, groups, datasets, datatypes, and dataspaces, which represent experimental observations, measurements, simulations, images, sensor outputs, and computational results.

Scientific data may be exposed through direct access using HDF5-based software, including command-line utilities, graphical viewers, and programming interfaces such as Python. However, privacy-sensitive information may also be inferred through examination of the low-level storage organization and auxiliary metadata embedded within the file to support portability, interoperability, and efficient data access. For example, string attributes visible through standard operating system utilities (e.g., the Unix strings command) may disclose dataset names, experimental identifiers, or other descriptive information. Similarly, inspection of dataset headers may reveal storage layout, chunk organization, compression methods, or data acquisition patterns (e.g., frame-based acquisition stored as chunked datasets). Examination of embedded strings may also disclose the use of external storage or references to datasets located in other HDF5 files.

It is important to distinguish between protecting access to data values and protecting information about data representation. Even when raw data are protected through mechanisms such as compression or encryption, information describing their storage organization, layout, or access methods may remain accessible and reveal sensitive contextual information about the underlying data or the processes that generated them.

Metadata Assets User-defined metadata often constitute one of the most privacy-sensitive asset classes in HDF5. The format was intentionally designed as a self-describing data model that allows extensive contextual information to be stored alongside scientific data. While HDF5 attributes are the most recognizable form of user-defined metadata, they represent only one category of descriptive information maintained within an HDF5 file.

User-defined metadata include all user-created HDF5 objects and object properties that organize, identify, or describe stored data. These include groups, datasets,

their associated datatypes and dataspaces, committed datatypes, and attributes attached to HDF5 objects. Each HDF5 attribute is itself a structured metadata object with a name, datatype, and dataspace. Additional metadata assets include file and object names (link names), committed datatype names, compound datatype member names, enumeration labels, object comments, and information stored in the user block. Collectively, these metadata describe the logical organization, semantics, and relationships of the stored data and frequently contain experiment-specific, operational, or organizational information.

Like scientific data, metadata assets may be exposed not only through HDF5 applications but also through direct examination of the binary file format. Inspection of file structures or extraction of embedded strings without using HDF5 software may reveal object names, descriptive attributes, storage relationships, user block contents, and other contextual information that could disclose sensitive information about the data, their origin, or the organization that produced them.

Structural Metadata Assets Privacy-sensitive information may also be inferred from the structural characteristics of HDF5 files. Structural metadata assets include user-defined organizational structures, such as the hierarchical arrangement of objects, dataset storage layouts (e.g., contiguous, chunked, compact, or external storage), filter pipelines, and relationships established through links, object references, region references, or virtual datasets (VDS). In addition, the HDF5 library maintains internal structural metadata that describe and manage these user-defined structures, including chunk indexing schemes, B-tree organizations, and other implementation-specific data structures.

Even when raw data are inaccessible, for example, because they are encrypted, compressed, stored externally, or protected by file-system access controls, structural metadata may disclose information about the purpose of the data, experimental design, workflow organization, relationships among datasets, or assumptions made by the generating application. Naming hierarchies, grouping strategies, storage layouts, and reference relationships may reveal semantic information about scientific activities or operational processes. Likewise, implementation-specific structures, such as chunk indexing formats or other internal metadata, may dis-

close characteristics of the HDF5 library version or software implementation used to create or process the file.

Like scientific data and user-defined metadata, structural metadata may be exposed through HDF5 applications or by direct examination of the binary file format without using HDF5 software.

Derived and Reconstructed Information Privacy-sensitive information in HDF5 may also arise through inference or reconstruction from combinations of available data and metadata. Derived assets include relationships inferred from multiple HDF5 datasets, external links, object references, region references, virtual dataset (VDS) mappings, or information obtained by combining HDF5 data with external sources. These relationships may reveal information that is not explicitly represented within any single dataset or file.

For example, correlating quality-control datasets with experimental observations, combining temporal and spatial measurements, or reconstructing relationships across linked HDF5 files may disclose sensitive characteristics that are not apparent when individual datasets are examined in isolation. Consequently, privacy-sensitive assets extend beyond explicitly stored data and metadata to include information that can be inferred through aggregation, correlation, reconstruction, or other forms of analysis.

Workflow and Applications Artifacts Privacy-sensitive assets in HDF5 extend beyond data and metadata stored within HDF5 files to include artifacts created by applications, processing workflows, and the surrounding software ecosystem. These artifacts include command outputs (e.g., h5dump output), diagnostic logs, temporary files, metadata snapshots, cached intermediate datasets, conversion artifacts, and outputs generated by plugins or connectors (e.g., binary files produced by an HDF5 VOL connector for another scientific data format).

Examples include VFD SWMR metadata snapshots, cached intermediate results, diagnostic logs, and metadata generated during translation between non-self-describing data formats and HDF5. Such artifacts may expose scientific data,

metadata, storage organization, workflow decisions, or implementation details beyond what is contained in the original HDF5 file. Because they are often created automatically, these artifacts may persist beyond their intended lifetime and remain unnoticed by users focused primarily on the scientific datasets themselves.

Scientific data processing workflows routinely enrich, restructure, annotate, and transform datasets to improve interoperability, reproducibility, usability, or analytical value. However, these processes may also introduce or preserve contextual information that was not intended for disclosure. Automatically generated metadata, intermediate processing products, and derived datasets may reveal experimental assumptions, workflow decisions, data provenance, software implementation details, or scientific semantics that were absent from the original data. Consequently, privacy exposure may arise not from malicious activity or unauthorized access, but as an unintended consequence of routine scientific data processing.

3.4. HDF5 Tools

Not strictly part of the core library, but the tools are an important part of the ecosystem and have their own set of issues.

3.4.1. Privacy exposure through HDF5 command-line tools

HDF5 command-line tools provide convenient mechanisms for managing HDF5 files. They can be used to display or extract raw data, and user-defined and structural HDF5 metadata (e.g., `h5dump`, `h5ls`, `h5debug`), edit raw data and attributes with `h5edit`, extract or aggregate data using `h5copy`, change characteristics of HDF5 objects, for example, remove or add data filters and change storage properties using `h5repack`. HDF5 command-line tools are routinely employed throughout the HDF5 data lifecycle. While the command-line tools are essential for working with HDF5 data, they are ultimate sources of unintended privacy exposure.

The HDF5 tools were designed to ease access to HDF5 data, to reveal information about the contents and organization of HDF5 files, and to manage that information

while avoiding steep learning curve of HDF5 programming. I.e., HDF5 command-line tools were designed to expose HDF5 assets. Depending on the tool and the selected options, they may display object names, attributes, datatype definitions, storage layouts, chunking parameters, filter pipelines, external storage locations, object references, virtual dataset mappings, and portions or all of the stored data to name a few. Even when users intend only to verify file integrity or inspect metadata, command output may disclose information that is considered sensitive in a particular context.

Privacy exposure is not limited to information displayed on the terminal by `h5dump` or `h5ls`. Command output is frequently redirected to files, incorporated into automated workflow logs, attached to bug reports, archived as diagnostic information, or shared. The output can be used by other tools (e.g., `h5import`) or applications to generate new data. Consequently, metadata and scientific data extracted by HDF5 tools may persist outside the original HDF5 file and become accessible to individuals who were never intended to access the underlying information.

Several command-line tools may also create new HDF5 files or transform existing ones. `h5repack` and `h5copy` may generate additional copies of data and/or aggregate data, modify storage layouts, remove or add data filtering; `h5edit` may modify and introduce new metadata, change raw data values and preserve information that was not expected to be retained. I.e., the tools can alter the privacy characteristics of the resulting files, particularly when they are incorporated into larger workflows, backups, or public data repositories.

Because HDF5 command-line tools are widely used in automated scripts, continuous integration systems, and scientific workflows, their privacy implications should be evaluated alongside those of the HDF5 library and HDF5-based applications. The privacy mitigation strategies discussed in the next section apply not only to the HDF5 library itself but also, and often to an even greater extent, to HDF5 command-line tools. This is because these tools are designed to expose HDF5 assets and to generate persistent outputs (e.g., new binary and text files).

The table below summarizes the assets exposed by the listed command-line tools.

Table 2 – Privacy exposure through HDF5 command-line tools

Tool	Exposed assets	Workflow artifacts	Privacy considerations
`h5dump`	Raw data; user-defined and structural metadata	Text dumps, redirected output, logs	Displays datasets, attributes, datatypes, storage layout, filter information, object references, and VDS mappings. May expose data stored in other HDF5 files through external links and Virtual Datasets (VDS). Output is frequently redirected to files or logs, creating persistent copies of sensitive information.
`h5ls`	User-defined and structural metadata (optionally raw data)	Terminal output, logs	Reveals object hierarchy, object names, datatypes, storage properties, links, and object relationships. With the `-d` option, also displays dataset values.
`h5repack`	Raw data; user-defined and structural metadata	New HDF5 file	Creates additional HDF5 copies while potentially modifying storage layout, filters, and metadata. The `--merge` option can aggregate datasets reachable through external links into a single file. May access non-HDF5 data through configured VFDs and VOL connectors.
`h5diff`	Raw data; user-defined metadata	Comparison reports	Reads and compares datasets and attributes. Reports may disclose sensitive values or structural differences.
`h5copy`	Raw data; user-defined and structural metadata	Additional HDF5 files	Copies objects within or between HDF5 files, increasing the number of persistent copies of sensitive information.
`h5stat`	Structural metadata	Statistics reports, logs	Reports object counts, storage statistics, chunk information, free space, and other implementation characteristics that may reveal dataset complexity or workflow properties.
`h5debug`	Structural metadata	Debug output	Displays low-level internal file structures, addresses, B-trees, heaps, chunk indexes, and other implementation metadata intended for debugging.
`h5watch`	Raw data; user-defined metadata	Monitoring output, logs	Monitors SWMR datasets and reports newly written data, timestamps, and update activity in real time.
`h5import`	Raw data; user-defined metadata	New HDF5 file	Imports external data into HDF5, creating additional copies while potentially preserving metadata and semantics from the source format.

Table 2 – Privacy exposure through HDF5 command-line tools, continued

Tool	Exposed assets	Workflow artifacts	Privacy considerations
`h5jam`/`h5unjam`	User block contents	Extracted or embedded files	Stores or extracts arbitrary information in the HDF5 user block, including scripts, XML documents, application metadata, or proprietary information.
`h5mkgrp`	Structural metadata	Modified HDF5 file	Creates or modifies the group hierarchy. Limited direct privacy exposure but changes descriptive metadata and namespace organization.
`h5edit`	Raw data; user-defined metadata	Modified HDF5 file	Creates or modifies attributes; modifies raw data. Limited direct privacy exposure but changes data and user-defined metadata.
`h5format_convert`	Raw data; structural metadata	Converted HDF5 file	Converts files between HDF5 format versions, creating additional copies while preserving or rewriting internal metadata structures.
`h5clear`	Structural metadata	Modified HDF5 file	Clears SWMR status and metadata cache information. Primarily affects recovery metadata and file consistency information.

Note: All HDF5 command-line tools are built on top of the HDF5 library and therefore inherit its capabilities. When VFDs or VOL connectors are configured, these tools may transparently access data stored in remote services, external storage systems, or non-HDF5 data sources exposed through connectors. Consequently, privacy exposure is not necessarily limited to the HDF5 file specified on the command line but may extend to the broader HDF5 ecosystem.

3.5. Audit Findings and Mitigation Priorities

3.5.1. Security and Safety

The blunt conclusion: HDF5's CVE history is mostly malformed-file parsing pressure against a complex binary metadata machine. The fix direction should not be "patch each crash." It should be systematic metadata validation at decode boundaries, explicit invariants for each on-disk structure, fuzz corpora tied to

invariants, and hard separation between parser acceptance and internal object construction.

Goal: HDF5 should be able to parse any byte stream as hostile input and either reject it cleanly or produce a validated internal object graph. Anything else is security debt.

Implementation program: define parser invariants first, harden decode boundaries, then make fuzzing, CI gates, and release governance enforce them.

The next priority is parser normalization: decode file bytes into a validated intermediate form before constructing internal objects, so malformed metadata cannot reach trusted C paths.

Our goal should be: *make malformed HDF5 files boring*. Every corrupted file should terminate in a controlled `H5E_BADVALUE`, `H5E_OVERFLOW`, `H5E_CORRUPT`, or `H5E_RESOURCE` path. No segfaults. No assertions. No leaks. No undefined behavior. No late-stage encode/flush crash from a bad decode.

The current CVE list includes many issues in `H5FS`, `H5C`, `H5O`, `H5T`, `H5Z`, and vector-copy paths, with repeated heap overflows, use-after-free, null dereference, and resource-consumption outcomes. HDF5's own format documentation explains why this happens: a file is a graph of groups, datasets, and named datatypes; on disk it is represented through lower-level structures such as the superblock, B-tree nodes, object headers, global heaps, local heaps, and free-space metadata. That means the library is parsing a dense graph of mutually referential, *attacker-controlled* metadata.

Priority 0: Stop closing CVEs as "fixed" unless the class is killed

Make this rule immediate:

If the fix is not a structural change that eliminates the root cause, then it is not a fix.

For every new CVE or crash:

Table 3 – Required artifacts for closing HDF5 CVEs or crash reports

Required artifact	Purpose
crashing input	becomes a permanent corpus seed
minimized input	used in quick PR regression
root-cause category	maps to a known invariant class
invariant update	prevents sibling bugs
sanitizer regression	proves no crash, no leak, no UAF
negative test	checks controlled failure code
reviewer checklist update	prevents reintroduction

This is the first cultural shift. A patch that only adds a local `if` is tolerated for embargoed fixes, but the issue remains open as a **class bug** until the invariant framework catches sibling cases.

Recent commits show the right direction, but too locally. One fix added a sanity check immediately after reading file-space page size from the file and used the same fuzzer to verify that the previous assertion became a corrupted-file error. [**commit_804f3ba**] Another hardened `H5C__reconstruct_cache_entry()` by adding buffer-size tracking, overflow checks, input validation, and safe cleanup. [**commit_3914bb7**] Those are exactly the moves to systematize.

Priority 1: Build an invariant registry for the file format

Create a versioned document and machine-readable metadata file, for example:

```
security/invariants/
  superblock.yml
  object_header.yml
  dataspace.yml
  datatype.yml
  layout.yml
```

```
filters.yml
heaps.yml
free_space.yml
links.yml
cache_image.yml
```

Each entry should define:

```yaml
id: H5O-CONT-001
component: object_header
structure: continuation_message
condition: continuation_chunk_size > 0
failure: H5E_CORRUPT
checked_at:
  - H5O__cont_decode
  - H5O__chunk_deserialize
fuzz_targets:
  - fuzz_object_header
cves:
  - CVE-2025-6856
  - CVE-2025-6816
```

Start with these invariant families.

Table 4 – Invariant families to enforce first

Area	Invariants to enforce first
superblock	offset size and length size are valid; base address, EOF address, driver info address, and root object address are inside the file or explicitly undefined; EOF does not wrap
object headers	message length fits inside chunk; continuation chunks are nonzero; continuations do not self-reference; continuations are acyclic; message flags match message type; unknown messages cannot be cast into known message structures
free-space meta-data	page size is nonzero and below max; section size fits page/file bounds; serialized section count matches decoded section count
heaps	heap object offset and size are inside heap; free-list blocks do not overlap; free-block list is acyclic; no allocation uses unchecked `count*size`

Table 4 – Invariant families to enforce first, continued

Area	Invariants to enforce first
dataspaces	rank is bounded; dimension product is checked; selected point count is checked; hyperslab stride, count, block, and start do not overflow
datatypes	storage size is nonzero; precision fits storage size; member offsets fit compound size; string length is bounded; vlen/ref types cannot escape validation
layouts	compact data size equals `dataspaceelements*datatypesize`; chunk dimensions are nonzero; chunk product is checked; chunk index references are inside file bounds
filters	parameter count is bounded; decompressed size is bounded; scale-offset precision is validated; filter output ratio cannot exceed configured resource limits
cache image	cache-entry count, address, size, and alignment are bounded; decoding consumes exactly the expected number of bytes; cleanup is safe after partial decode
tools	string rendering is length-bounded; conversion tools never trust input dimensions; tool-only parsers use the same checked arithmetic wrappers as the library

This registry becomes the source of truth for code review, tests, fuzzing, and documentation.

Priority 2: Introduce a two-phase parser boundary

Right now, too many paths appear to do this:

```
file bytes -> native internal object
           -> later code assumes validity
```

Change the architecture to:

```
file bytes -> bounded raw decode
           -> semantic validation
           -> native object construction
```

Concrete rule: *No public or internal runtime object is constructed from file metadata until the corresponding validator has passed.*

Implement the pattern by component:

```
herr_t H5O_msg_decode_raw(const uint8_t *buf, size_t buf_size,
↪  H5O_msg_raw_t *raw);
herr_t H5O_msg_validate(const H5F_t *f, const H5O_msg_raw_t *raw,
↪  H5O_validation_ctx_t *ctx);
herr_t H5O_msg_construct(const H5O_msg_raw_t *raw, H5O_msg_t *out);
```

Do not let decode functions allocate complex structures before basic bounds are known. A raw decoded object should hold values, offsets, and spans, not trusted pointers into long-lived internal state.

All decode functions that consume file bytes should take one of these forms:

```
herr_t decode(const H5F_t *f, const uint8_t *buf, size_t buf_size,
↪  out_t *out);

herr_t decode_cursor(H5_decode_cursor_t *cur, out_t *out);
```

Ban this pattern in security-sensitive decode code:

```
const uint8_t **pp
```

unless there is also an explicit `p_end` or remaining-size counter. The cache-image fix already moved in this direction by adding a buffer-size argument and bounds checks. [**commit_3914bb7**]

Priority 3: Centralize checked arithmetic and allocation

Most parser bugs become exploitable when metadata reaches allocation, copy, or pointer arithmetic. Add a small, mandatory checked-arithmetic layer.

Example API:

```
herr_t H5_checked_add_hsize(hsize_t a, hsize_t b, hsize_t *out);
herr_t H5_checked_mul_hsize(hsize_t a, hsize_t b, hsize_t *out);
herr_t H5_checked_addr_add(haddr_t base, hsize_t len, haddr_t eoa,
↪  haddr_t *end);
herr_t H5_checked_alloc_size(size_t count, size_t elem_size, size_t
↪  *bytes);
```

Then make these rules review-blocking:

Table 5 – Review-blocking parser arithmetic and allocation patterns

Ban	Replacement
`malloc(n*size)`	`H5MM_calloc_checked(n,size)`
`addr+size`	`H5_checked_addr_add(addr, size,eoa,&end)`
`memcpy(dst,src,n)` from file-derived size	`H5_checked_memcpy(dst,dst_size,src,src_size,n)`
pointer increment without `p_end`	cursor-based decode
raw recursion through object graph	bounded traversal context

The 2024 security commit added multiplication-overflow checks for dataset storage size and checks against file bounds. [**commit_ce53bc0**] That pattern should become a library-wide rule, not a local repair.

Priority 4: Create security profiles for file opening

Add explicit modes. Do not rely on users to infer safe behavior.

```
typedef enum {
    H5_SECURITY_PROFILE_LEGACY,
    H5_SECURITY_PROFILE_TRUSTED_FAST,
    H5_SECURITY_PROFILE_UNTRUSTED_STRICT,
    H5_SECURITY_PROFILE_FORENSIC
} H5_security_profile_t;
```

Suggested semantics:

Table 6 – Security profile semantics

Profile	Behavior
`legacy`	current compatibility bias
`trusted_fast`	normal validation, generous resource limits
`untrusted_strict`	strict metadata validation, plugin loading disabled unless allowlisted, resource limits enforced
`forensic`	never repairs silently, reports all structural anomalies, best-effort inspection without unsafe traversal

Expose it through file-access property lists:

```
H5Pset_security_profile(fapl, H5_SECURITY_PROFILE_UNTRUSTED_STRICT);
H5Pset_max_metadata_bytes(fapl, 256 * 1024 * 1024);
H5Pset_max_object_header_chunks(fapl, 1024);
H5Pset_max_btree_depth(fapl, 64);
H5Pset_max_filter_expansion_ratio(fapl, 100);
H5Pset_plugin_policy(fapl, H5_PLUGIN_POLICY_DISABLED);
```

This matters because HDF5 supports dynamically loaded plugins, and the runtime API can enable or disable plugin classes; the docs also state that `HDF5_PLUGIN_PRELOAD=::` disables all dynamic plugins. Turn that capability into a first-class security policy, not an environment-variable trick.

Priority 5: Fuzz by structure, not just by whole-file crashes

Whole-file fuzzing is necessary but insufficient. It finds symptoms. Structure-aware fuzzing finds families.

Build fuzz targets in this order:

Table 7 – Structure-aware fuzzing priorities

Order	Fuzz target	Why first
1	object-header message decoder	many paths flow through it
2	datatype decoder and converter	high exploitability and complex nested metadata
3	dataspace and selection decoder	rank, count, hyperslab, and point math
4	layout and chunk index decoder	storage-size, address, and chunk-product risks
5	free-space and local/global heap decoders	repeated historical overflow and list-corruption class
6	filter pipeline and scale-offset filter	compression/decompression boundary risk
7	metadata cache image reconstruction	high complexity and recent CVE pattern
8	`h5dump`/`h5repack` rendering paths	tool paths often bypass library assumptions

Use three fuzzing layers:

Table 8 – Fuzzing layers

Layer	Description
byte-level fuzzing	current randomized mutation against APIs
structure-aware mutation	valid superblock with corrupted object headers, valid object header with bad continuation, valid datatype with bad member offset
invariant-guided generation	generate files that violate exactly one invariant at a time

OSS-Fuzz already supports libFuzzer, AFL++, Honggfuzz, Centipede, sanitizers, and distributed execution for C/C++ projects. Use OSS-Fuzz for long-running public coverage and ClusterFuzzLite or local CI for PR-level checks.

Table 9 – Fuzzing acceptance criteria

Stage	Gate
PR	run minimized CVE corpus under `ASan`/`UBSan`/`LSan`
nightly	run structure fuzzers for core parsers
weekly	report coverage by component, not just total line coverage
release candidate	72-hour fuzz soak on changed parser areas
CVE closure	crashing input added to corpus and mapped to invariant

Priority 6: Make malformed-input regression cheap

Create this repository layout:

```
test/security/
  corpus/
    cve/
      CVE-2025-7069/
        input.h5
```

```
        expected.yml
      CVE-2025-6269/
        input.h5
        expected.yml
    invariants/
      H5O-CONT-001/
      H5T-SIZE-002/
  harnesses/
    fuzz_object_header.c
    fuzz_datatype.c
    fuzz_dataspace.c
    fuzz_layout.c
    fuzz_filters.c
  expected_errors/
    corrupt_file.yml
```

Each expected.yml should include:

```yaml
expected:
  exit: controlled_error
  allowed_errors:
    - H5E_CORRUPT
    - H5E_BADVALUE
    - H5E_OVERFLOW
forbidden:
  - SIGSEGV
  - SIGABRT
  - leak
  - use_after_free
  - timeout
invariant:
  - H5FS-PAGE-001
```

This gives maintainers a fast way to prove that old crashes remain dead.

Priority 7: Add a security-review checklist that blocks risky parser changes

Every PR touching decode, encode, object lifecycle, filters, tools, or plugin loading should answer:

Table 10 – Security-review checklist for parser-adjacent changes

Question	Required answer
Does this read file-derived bytes?	list buffer and bounds source
Does this allocate using file-derived values?	show checked multiplication
Does this copy memory using file-derived sizes?	show source and destination bounds
Does this traverse links, chunks, heaps, B-trees, or continuations?	show cycle and depth bounds
Does this construct native objects before validation?	explain why or refactor
Does this add or change cleanup paths?	show partial-init safety
Does this invoke plugins or filters?	show policy interaction
Does this alter error handling?	show malformed-input test

Add automated enforcement where possible:

```
fail PR if new decode function lacks size parameter
fail PR if malloc(count * size) appears outside checked wrapper
fail PR if memcpy appears in parser code without checked wrapper annotatio
fail PR if new parser file lacks fuzz target registration
fail PR if new object traversal lacks max-depth or visited-set logic
```

This will annoy contributors at first. Good. That friction is cheaper than post-release CVEs.

Priority 8: Separate compatibility from safety

HDF5's backward compatibility is valuable, but it creates a security trap: old, permissive parsing keeps malformed legacy structures alive.

Implement this policy:

Table 11 – Compatibility and safety profile policy

Case	Behavior
valid legacy file	accepted
ambiguous legacy construct	accepted only in `legacy` or `trusted_fast`; warning in `forensic`; rejected in `untrusted_strict`
structurally invalid file that old versions tolerated	rejected in all non-legacy profiles
parser repair needed	repair only through explicit tool, never silently during normal open
unknown message with unsafe flags	rejected unless profile explicitly allows preservation

The point is not to break legitimate archives. The point is to stop treating corrupted metadata as compatibility.

Priority 9: Harden tools as separate attack surfaces

Do not assume `h5dump`, `h5repack`, converters, and test utilities are less important. They are often the first thing users run on untrusted files.

Actions:

Table 12 – Tool hardening actions

Tool class	Change
inspection tools	use `H5_SECURITY_PROFILE_FORENSIC` by default

Table 12 – Tool hardening actions, continued

Tool class	Change
conversion tools	use `H5_SECURITY_PROFILE_UNTRUSTED_STRICT` by default
rendering paths	bounded strings only
legacy converters	disable by default; build behind explicit `HDF5_ENABLE_LEGACY_UNSAFE_TOOLS=ON`
plugin-dependent tools	warn or require allowlist

A dangerous file should not become more dangerous because the user tried to inspect it.

Priority 10: Plugin policy and signed extensions

Plugins are a different risk class. They are executable code loaded at runtime. HDF5's plugin system supports dynamically loaded filter plugins and lets users implement custom filters. That is useful, but it must not be invisible in untrusted mode.

Implement:

```
default profile:
  trusted_fast: existing plugin behavior
  untrusted_strict: plugins disabled unless allowlisted
  forensic: plugins disabled; report missing filter
```

Add a plugin manifest:

```
plugin_id: 32008
name: blosc
version: 1.0.0
sha256: ...
signer: ...
allowed_profiles:
```

```
  - trusted_fast
  - untrusted_strict
capabilities:
  - filter_decode
limits:
  max_expansion_ratio: 100
  max_alloc_bytes: 1073741824
```

Do not start with a perfect PKI. Start with allowlisted hashes and paths. Then add signatures. The signed-plugin work matters, but without resource limits a signed buggy plugin can still crash the process.

Priority 11: Supply-chain controls after parser controls

Supply-chain work matters, but it is second-order relative to malformed-file parsing. Do it in parallel only after the parser program has owners.

Minimum controls:

Table 13 – Minimum supply-chain controls

Control	Target
OpenSSF Scorecard	weekly scheduled run, alerts in security tab
CodeQL	required for PRs touching C/C++ parser surfaces
pinned GitHub Actions	no floating third-party action refs
SBOM	release artifacts include source and binary SBOMs
signed release artifacts	signatures plus checksums
SLSA provenance	release builds produce verifiable provenance

OpenSSF Scorecard has an official GitHub Action, and public repositories can use it without cost. The SLSA GitHub generator provides tooling for SLSA provenance on GitHub Actions, including provenance that users can verify to understand how an artifact was built.

Priority 12: Metrics that force the right behavior

Use metrics that measure class elimination, not activity.

Table 14 – Security program metrics

Metric	Target
CVE corpus pass rate under ASan/UBSan/LSan	100%
parser decode functions with explicit buffer size or cursor	100%
parser allocations through checked wrappers	100%
parser memcpy/memmove through checked wrappers or reviewed exemption	100%
high-risk structures with invariant files	100%
high-risk structures with fuzz target	100%
new CVEs that add invariant updates	100%
time from new crash to minimized corpus seed	under 48 hours
time from minimized seed to invariant classification	under 5 business days
sanitizer-clean nightly fuzz runs	sustained
untrusted profile test coverage	every release

Track this in a public dashboard. Not because dashboards fix bugs. They stop the team from pretending that "more tests" is a plan.

3.5.2. Privacy

In this section we summarize privacy threats in HDF5 based on the audit findings and describe mitigation strategies and their priorities.

Overview of Privacy Threat Categories: The HDF5 Privacy Exposure Model [**PVEM**] identifies seven categories of privacy threats that describe the principal mechanisms through which sensitive information may be unintentionally exposed in HDF5. Privacy threats primarily arise during normal scientific data creation,

processing, sharing, and preservation. They are a consequence of HDF5's self-describing data model, rich metadata capabilities, extensible software architecture, broad interoperability across scientific applications, and prolonged data life cycle.

Several threat categories involve direct disclosure of information stored within HDF5 files. Metadata Exposure (P1) occurs when descriptive information, such as object names, attributes, datatype member names, or other user-defined metadata, reveals sensitive contextual information even when the underlying scientific data are protected. Structural Metadata Exposure (P2) arises from examination of HDF5's internal storage structures, including file layout, storage organization, and implementation metadata, which may disclose characteristics of the stored data without requiring HDF5 software. Raw Data Exposure (P3) addresses situations in which information about scientific data becomes observable through storage patterns, embedded textual descriptions, or other implementation artifacts rather than through intended data access.

Another group of threats results from routine scientific data processing. Unintended Data Alteration or Inclusion (P4) occurs when software automatically generates, expands, or preserves additional information during data conversion or processing. Data Aggregation and Correlation (P5) recognizes that individually non-sensitive datasets may collectively reveal sensitive information when combined or interpreted together. Persistent Data Beyond Intended Usage (P6) addresses privacy risks associated with temporary files, metadata snapshots, cached datasets, archived copies, and other artifacts that remain accessible after their intended purpose has ended.

The remaining threat category Misplaced Trust in Privacy by format (P7) reflects broader characteristics of modern HDF5 ecosystems. Examples include:

- Misinterpretation of data privacy in HDF5 format leads to the possibility of sensitive information disclosure through metadata, structural information, or inference.
- Conversion to HDF5 highlights that transforming legacy scientific formats into HDF5 often increases accessibility and interoperability by making implicit information explicit, but this additional semantic information may itself introduce new privacy risks.

- Loss of Data Control describes situations in which data movement, replication, backup, cloud migration, or transfer across organizational boundaries weakens the original privacy protections and governance mechanisms applied to the data.

Although presented as distinct categories, these threats frequently occur together. A single scientific workflow may enrich metadata during format conversion, aggregate datasets from multiple sources, generate temporary processing artifacts, and replicate results across multiple storage systems. Consequently, privacy risks should be evaluated across the complete scientific data lifecycle rather than by considering individual HDF5 files in isolation.

Mitigation Priorities. Because HDF5 was designed to maximize scientific data sharing, portability, interoperability, and long-term usability rather than enforce privacy, no single mechanism can eliminate all privacy risks. Effective privacy protection therefore requires a multi-facet approach that considers the entire scientific data lifecycle, including data creation, processing, storage, sharing, transfer, and archival. Privacy should be treated as a property of the complete HDF5 ecosystem including applications, plugins, workflows, storage systems, and organizational practices rather than solely of the HDF5 file format. Here we list mitigation strategies and their priorities.

Priority 1: *Privacy exposure awareness.* Scientists, software developers, data stewards, and infrastructure providers should recognize that HDF5's self-describing nature inherently exposes contextual information in many ways. Privacy reviews should therefore evaluate not only scientific datasets, but also metadata, structural organization, workflow artifacts, derived products, and information that may be inferred through aggregation or correlation. Reviews should also take into consideration possible privacy exposure during data lifecycle.

Priority 2: *Privacy exposure through HDF5 tools and plugins awareness.* The HDF5 ecosystem extends beyond the core HDF5 software (i.e., file format

and the library), and includes a diverse collection of command-line utilities, visualization tools, analysis libraries, VOL connectors, VFDs, filters, and third-party applications. These components frequently generate additional outputs such as logs, diagnostic messages, temporary files, metadata snapshots, converted datasets, and cached intermediate results that may expose sensitive information beyond the contents of the original HDF5 file. Furthermore, plugins may introduce new metadata, or transform data in ways that alter their privacy characteristics. Consequently, privacy assessments should evaluate not only HDF5 files but also the behavior of the software ecosystem that processes them. Developers should document the privacy implications of their tools, minimize unnecessary generation of sensitive artifacts, and provide users with mechanisms to control logging, metadata generation, and retention of intermediate products.

Priority 3: ***Privacy-aware system design.*** HDF5 software systems should be built with data privacy in mind. Applications should minimize collection and retention of unnecessary metadata, avoid embedding sensitive information in object names or descriptive attributes, remove temporary processing artifacts when they are no longer needed, and carefully evaluate automatically generated metadata introduced during data conversion or interoperability. Workflow designers should review intermediate products, logs, cached datasets, and derived files before publication or long-term archival.

Priority 4: ***Data encryption as privacy control.*** Encryption represents an important but limited privacy control.

Currently, HDF5 doesn't support data encryption that allows I/O on encrypted data, effectively disabling sensitive data exposure during data transfer and storage. Users may implement custom raw data encryption and partial metadata encryption (i.e., names of the objects) using existing HDF5 filtering mechanism. Such approach still leaves user-defined metadata (e.g., attributes) and structural metadata unencrypted. Information that remains unencrypted as application-specific workflow artifacts, temporary files, system logs, or derived datasets may continue to disclose sensitive information.

HDF5 encryption feature that properly implements encryption of all data stored in HDF5 while allowing partial I/O on encrypted data is currently under development. However, encryption does not eliminate all HDF5 privacy risks. Once encrypted data are decrypted for legitimate processing, applications, plugins, caches, diagnostic logs, and intermediate workflow products may again become sources of unintended exposure. Encryption therefore reduces the attack surface but does not address inference, aggregation, excessive metadata generation, data retention, or loss of governance. Encryption also complicates workflows and may lead to data loss if on some stage of data life cycle encryption keys become unavailable.

Therefore, encryption should be viewed as one component of a comprehensive privacy strategy rather than as a complete solution. Effective protection requires combining encryption with careful metadata management, lifecycle-aware data governance, privacy-aware application design, access controls provided by the underlying computing environment, and education of users regarding the unique privacy characteristics of HDF5-based scientific data systems.

4. HDF5 Library Extensions

4.1. Overview

Supply chain-related issues are discussed in more detail in Section 7.

4.2. Virtual Object Layer (VOL) Connectors

4.2.1. Privacy Risks for VOL Connectors

HDF5 VOL connectors present one of the most significant privacy challenges in the HDF5 ecosystem. When a VOL connector (or a stack of VOL connectors) is used, data privacy can no longer be evaluated by examining only the HDF5 file, its lifecycle, and the workflows in which it is used. Instead, the privacy boundary extends to the entire data ecosystem, including all underlying data sources, connector-generated data and metadata, caches, authentication credentials, network communications, and the dynamically loaded plugin code itself. Consequently, any comprehensive HDF5 privacy model must account for these components, their interactions, and the trust relationships they establish.

We identified the following risks when using HDF5 VOL connectors.

External Data Sources (P8 Ecosystem Exposure)
Users traditionally assume that an HDF5 file contains all of the data being accessed and that inspecting or protecting the file is sufficient to understand and secure the complete dataset. However, this assumption no longer holds when HDF5 VOL connectors are used. A VOL connector can transparently retrieve and integrate data from numerous external sources while presenting it as a single HDF5 file. These sources may include cloud object stores, databases, remote servers accessed through REST or other network services, live data streams, or files in non-HDF5 formats. In addition, connectors may generate derived data, metadata, or virtual objects that do not physically exist in the underlying storage. As a result, users may be unaware of the true origin, location, ownership, and privacy policies

governing the data being accessed, making it more difficult to assess privacy risks and apply appropriate protections.

Transparent Data Aggregation (P3 Derived Information, P8 Ecosystem Exposure)

A VOL connector can transparently aggregate data from multiple HDF5 files, databases, cloud object stores, external file formats, and live data streams into a single logical HDF5 dataset. For example, a connector could combine data from a geospatial database, near real-time weather and natural disaster feeds, and property assessment records to produce an HDF5 dataset for analyzing the potential impact of natural disasters on property values. From the application's perspective, all of these heterogeneous data sources appear as a single HDF5 dataset, potentially obscuring the number, nature, and privacy policies of the underlying data sources. Such transparent aggregation increases the risk of unintended disclosure because data originating from independent sources with different ownership, access controls, or privacy policies may be analyzed or shared as a single integrated dataset without the user's explicit awareness.

Access to non-HDF5 data (P8 Ecosystem Exposure)

VOL connectors can access data in other binary formats, for example TIFF/GeoTIFF, BUFR, GRIB2, and CDF, and special storage systems, for example, DAOS. Therefore, the privacy implications of these formats and storage systems become HDF5 privacy issues.

Generated metadata (P1 Metadata Exposure, P3 Derived Information)

VOL connectors may generate their own metadata that in turn may reveal, for example, dataset relationships, storage locations, URLs, cloud bucket names, and provenance. VOL connectors may generate additional metadata; for example, BUFR and GRIB2 VOL connectors generate file inventories revealing file organization, create derived attributes and dimension scales even when raw data remain protected.

Network Workflow Artifacts (P6 Workflow Artifacts)

HDF5 VOL connectors that access remote data sources generate network workflow artifacts as part of normal operation. These artifacts may include HTTP request and response headers, DNS queries and logs, proxy logs, browser or client history, authentication records, local caches, temporary downloads, and other network-related metadata retained on local or remote systems. Such artifacts can create a detailed record of the datasets accessed, potentially exposing sensitive information even when the underlying data remain protected. Consequently, HDF5 VOL connectors that rely on network communications inherit many of the privacy risks associated with networked systems and should be evaluated within a broader network privacy and security model.

Caching (P2 Raw Data Exposure, P3 Derived Information

HDF5 VOL connectors may cache all sorts of data (metadata, downloaded raw data, authentication information, etc.) that remain after execution and may be shared, copied, and archived. A user may not be even aware of VOL's artifacts and leaked data.

Provenance leakage (P1 Metadata Exposure, P3 Derived Information, P8 Ecosystem Exposure

VOL connectors may reveal provenance information, e.g., data origin, timestamps, organizational structure, modification history.

Policy Bypass (P8 Ecosystem Exposure)

HDF5 applications are generally privacy-policy agnostic and have no inherent awareness of the privacy policies governing the data they access. When VOL connectors are used, the same application may transparently access data originating from multiple sources, each subject to different ownership, access controls, retention requirements, or privacy regulations. Because the application interacts only with the HDF5 API, it may unknowingly combine, process, or redistribute data under incompatible privacy policies, effectively bypassing policy boundaries that would otherwise apply to the individual data sources.

Plugin Trust (P7 Trust adn Supply Chain)

HDF5 VOL connectors are dynamically loaded plugins that execute external code within the application's process. As a result, a connector has direct access to the data flowing through the HDF5 API and can inspect, copy, modify, cache, export, or share that data before returning it to the application. A malicious or untrusted VOL connector could therefore compromise the confidentiality, integrity, or privacy of the data without the application's knowledge. Consequently, VOL connectors should be obtained from trusted sources, verified before deployment, and included within the application's overall trust and privacy model.

Table 15 – Primary privacy mechanisms for VOL connector privacy risks

Risk	**Primary Privacy Mechanism**
External Data Sources	P8 Ecosystem Exposure
Transparent Data Aggregation	P3 Derived Information P8 Ecosystem Exposure
Access to non-HDF5 Data	P8 Ecosystem Exposure
Generated Metadata	P1 Metadata Exposure P3 Derived Information
Network Workflow Artifacts	P6 Workflow Artifacts
Caching	P2 Raw Data Exposure P3 Derived Information
Provenance Leakage	P1 Metadata Exposure P3 Derived Information P8 Ecosystem Exposure
Policy Bypass	P8 Ecosystem Exposure
Plugin Trust	P7 Trust and Supply Chain

4.3. Filters Plugins

4.3.1. Privacy Risks for Filters Plugins

immediately before storage and immediately after retrieval. As a result, the privacy boundary extends beyond the HDF5 file itself and the application that

uses the data. Filter plugins execute external code, may create temporary plaintext copies of sensitive data, generate intermediate artifacts, maintain internal state, or transmit information outside the application. Consequently, evaluating privacy risks requires considering not only the stored HDF5 file but also the behavior of the filter plugins, their execution environment, and any artifacts they produce during data processing.

We identified the following major risks when using HDF5 filters.

Metadata leakage via filters identifiers (P1 Metadata Exposure)

The filter pipeline is stored in the file as HDF5 metadata in the object header. Filter identifier and its parameters may reveal security mechanisms, organizational standards, software ecosystem, application domain, etc.

Data characteristics leakage via filter parameters (P1 Metadata Exposure, P3 Derived Information)

Filter parameters such as compression level, block sizes, scale-offset parameters, etc. may reveal expected precision, data resolution, floating-point accuracy, acceptable information loss. Usage of lossy filters shows acceptable data approximation, expected error bounds, and precision requirements. Such data characteristics can give insights into assumptions under which data was collected and characteristics of experimental setup.

Sizes of compressed chunks reveal data characteristics (P1 Metadata Exposure, P3 Derived Information)

Chunk sizes are metadata and can be examined without access to data. Different sizes of compressed chunks may reveal sparsity, repeated patterns, data randomness, and empty regions to name a few.

Filters operate on plaintext data buffers (P2 Raw Data Exposure)

Before data compression or encryption is applied filters have access to data. Mali-

cious filter may have access to data before specialized compression or encryption are applied to protect data.

Filters may persist data outside HDF5 (P2 Raw Data Exposure, P6 Workflow Artifacts)
Filters may allocate memory buffers that may remain in memory, or paged to disk, appear in crash dumps. This sensitive data may persist outside the HDF5 file.

Plugin trust (P7 Trust and Supply Chain)
As HDF5 VOL connectors, HDF5 filters are dynamically loaded plugins that execute external code within the application's process. If the filter is malicious, it can do anything before passing data to application or to storage, for example, inspect memory, write logs, copy decoded data and send it elsewhere or retain it. Untrusted filter plugin could compromise data confidentiality, integrity and privacy without application's knowledge.

Plugin search path (P7 Trust and Supply Chain)
Dynamic filters are loaded from directories specified by environment variables while the HDF5 library is searching for the requested plugin. Such process may result in loading a malicious filter. On different systems application may load different plugins. Thus, it becomes a supply-chain problem.

Filters may generate derived information (P3 Derived Information). Since filters have access to data they can generate and store any type of derived information, e.g., statistics, patterns, sparsity characteristics, etc. The derived information may reveal sensitive information about data.

The table below provides primary privacy mechanism for built-in filters and several popular third-party filters.

Table 16 – Primary privacy mechanisms associated with HDF5 filter plugins

HDF5 Filter	Primary Privacy Categories	Representative Privacy Considerations
Deflate (GZIP)	P1 Metadata Exposure P3 Derived Information	Compression method, compression level, and resulting chunk sizes may reveal data entropy, sparsity, and redundancy.
Shuffle	P3 Derived Information	Byte reordering itself is reversible, but its use together with compression may reveal characteristics of the underlying numeric representation and data organization.
Fletcher32	P1 Metadata Exposure	Checksum metadata reveals integrity protection is used but generally introduces minimal additional privacy risk.
N-bit	P1 Metadata Exposure P3 Derived Information	Bit-width reduction reveals effective numeric precision and expected value ranges, providing insight into experimental accuracy and measurement characteristics.
Scale-Offset	P1 Metadata Exposure P3 Derived Information	Scale factors and offsets reveal expected precision, acceptable quantization error, and data resolution.
ZFP	P1 Metadata Exposure P3 Derived Information	Compression mode (fixed rate, fixed precision, or fixed accuracy) and associated parameters reveal acceptable information loss and numerical accuracy requirements.
SZ / SZ3	P1 Metadata Exposure P3 Derived Information	Error bounds expose tolerated approximation error and characteristics of scientific datasets.
Blosc	P1 Metadata Exposure P3 Derived Information	Compression algorithm, block size, and shuffle/bitshuffle options reveal storage optimization choices and data organization.
Zstandard (Zstd)	P1 Metadata Exposure P3 Derived Information	Compression level and resulting compressed sizes may reveal entropy and structural properties of datasets.
LZ4	P1 Metadata Exposure P3 Derived Information	Fast compression with modest ratios may reveal data compressibility through chunk sizes.
BZIP2	P1 Metadata Exposure P3 Derived Information	Compression characteristics and block sizes may expose redundancy and data structure.

Table 16 – Primary privacy mechanisms associated with HDF5 filter plugins, continued

HDF5 Filter	Primary Privacy Categories	Representative Privacy Considerations
All Dynamic Filters	P2 Raw Data Exposure P6 Workflow Artifacts P7 Trust and Supply Chain	Filters operate on plaintext data before writing or after reading. Dynamically loaded plugins may inspect, cache, copy, or export sensitive data and may generate temporary buffers, logs, or crash artifacts.

4.4. Virtual File Drivers (VFD)

4.4.1. Privacy Risks for VFDs

HDF5 VFDs manage how actual bytes are stored and retrieved. While VFDs perform I/O and do not change logical HDF5 file and data model they may reveal privacy issues related to data movement, data storage, and application's I/O characteristics.

We identified the following major risks when using HDF5 VFDs.

External Data Sources (P8 Ecosystem Exposure)

Unlike traditional file I/O, Virtual File Drivers may transparently store and retrieve HDF5 data from cloud object stores, distributed file systems, network storage, memory-resident files, or multiple physical storage devices. Consequently, users and system administrators may incorrectly assume where sensitive information resides and which security policies, regulatory requirements, or administrative controls apply. The privacy boundary therefore extends beyond the HDF5 file to the entire storage ecosystem.

Multiple Copies of Data (P2 Raw Data Exposure, P6 Workflow Artifacts)

Some VFDs create additional copies of HDF5 data without the application's explicit awareness. These copies may exist in replicated files, metadata journals, backing stores, memory dumps, swap files, or mirrored storage systems. Because

these copies may persist beyond the lifetime of the primary HDF5 file, sensitive information can remain recoverable even after the original file has been deleted or relocated.

Storage Metadata Exposure (P1 Metadata Exposure)

Although VFDs do not understand HDF5 objects, they manage physical storage and therefore may expose low-level storage metadata, including physical offsets, allocation sizes, storage layouts, and file organization. Such information can reveal characteristics of datasets, storage architecture, and application behavior even without accessing the logical HDF5 objects.

Access Pattern Leakage (P3 Derived Information)

Storage activity observed by the operating system, distributed file system, storage servers, or cloud infrastructure may reveal application behavior. Access frequency, read/write ratios, sequential versus random I/O, update frequency, and accessed file regions can reveal properties of the underlying datasets, application algorithms, or user activity. These observations enable inference of information that is not explicitly stored within the HDF5 file.

Workflow Artifacts (P6 Workflow Artifacts)

VFDs may generate artifacts during normal operation, including metadata journals, temporary files, swap files, memory images, crash dumps, replicated storage objects, and cloud access logs. These artifacts frequently persist outside the HDF5 file itself and may retain sensitive information or evidence of application activity.

Plugin Trust (P7 Trust and Supply Chain)

Dynamically loadable Virtual File Drivers execute external code within the application's process. A malicious or compromised VFD can inspect, modify, retain, duplicate, or export data before writing it to storage or returning it to the application. Consequently, VFDs should be obtained only from trusted sources and considered part of the application's overall trust and privacy model.

Table 17 – Major privacy risks associated with HDF5 Virtual File Drivers

HDF5 VFD	Privacy Categories	Representative Privacy Risks
SEC2	P2 Raw Data Exposure	Disk blocks allocated for new HDF5 files may contain residual information if the underlying storage is not securely initialized or overwritten.
STDIO	P2 Raw Data Exposure	Similar to the SEC2 driver, persistent storage may expose residual data remaining on reused disk blocks.
Core	P2 Raw Data Exposure P6 Workflow Artifacts	Stores the entire HDF5 file in memory. Sensitive information may persist in process memory, swap space, or operating-system crash dumps. When the backing-store option is enabled, the in-memory file is written to persistent storage upon closing.
Family	P2 Raw Data Exposure	Splits a logical HDF5 file into multiple physical files. Misconfigured permissions or incomplete deletion of family members may expose portions of the dataset.
Split	P1 Metadata Exposure P2 Raw Data Exposure P8 Ecosystem Exposure	Stores metadata and raw data in separate files, which may have different permissions or reside on different storage systems. The driver also creates multiple physical copies of file information.
Multi	P1 Metadata Exposure P2 Raw Data Exposure P8 Ecosystem Exposure	Stores different HDF5 memory types in separate files. Independent storage locations and permissions increase the risk of metadata and raw data disclosure.
Mirror	P2 Raw Data Exposure P7 Trust and Supply Chain P8 Ecosystem Exposure	Replicates HDF5 data to a remote mirror service. Privacy depends on secure communication, trusted mirror daemons, authentication, and protection of replicated copies.
Log (VFD SWMR)	P1 Metadata Exposure P6 Workflow Artifacts	Metadata journaling creates persistent journal files containing metadata snapshots for crash recovery, leaving additional workflow artifacts outside the HDF5 file.

Table 17 – Major privacy risks associated with HDF5 Virtual File Drivers, continued

HDF5 VFD	Privacy Categories	Representative Privacy Risks
MPI-I/O	P1 Metadata Exposure P2 Raw Data Exposure P8 Ecosystem Exposure	Transfers metadata and raw data through the underlying HPC communication infrastructure. Unless protected by lower-layer mechanisms, network traffic may expose sensitive information.
Subfiling	P1 Metadata Exposure P2 Raw Data Exposure P8 Ecosystem Exposure	Divides a logical HDF5 file into multiple subfiles distributed across storage devices. Fragmentation and independently managed files increase the risk of unauthorized disclosure through misconfigured access controls.
Onion	P1 Metadata Exposure P2 Raw Data Exposure P3 Derived Information	Maintains historical versions of HDF5 files, allowing recovery of deleted or overwritten data and reconstruction of metadata evolution, provenance, and workflow history.
ROS3	P6 Workflow Artifacts P8 Ecosystem Exposure	Accesses HDF5 files stored in S3-compatible object stores. HTTP requests, DNS queries, cloud access logs, and HTTP Range requests create network artifacts and extend the privacy boundary to cloud infrastructure.

4.5. Audit Findings and Mitigation Priorities

5. HDF5 Wrappers and Language Bindings

5.1. Overview

Complete:

- C#
- Java
- Python
- JavaScript
- Rust

Partials…

5.2. Python

- CVE-2025-995 [**cve-2025-9905**]
- CVE-2026-0897 [**cve-2026-0897**]
- CVE-2026-1669 [**cve-2026-1669**]
- CVE-2026-2492 [**cve-2026-2492**]

5.3. Java

5.4. Audit Findings and Mitigation Priorities

6. Applications using HDF5

Applications treat HDF5 as inert data, but HDF5 files can carry behavior-relevant metadata: paths, external references, huge declared shapes, filter/plugin requirements, serialized application objects, and model-layer definitions. That creates integration bugs even when the HDF5 library is behaving as designed.

The HDF Group's security policy [**hdf5-sec-pol**] already attempts the right scope distinction: core libhdf5, official tools, Remote Code Execution (RCE) via parsing/API abuse, and supply-chain compromise are in scope; third-party plugins and application-level usage are usually triaged or referred to their maintainers.

Table 18 – Common application-level HDF5 failure families

Family	Typical failure	Examples	Why it is common
malformed file metadata reaches C memory code	buffer overflow, over-read, UAF, double-free, null dereference, divide-by-zero	many HDFGroup records, including 2024 and 2025 issues in object headers, datatypes, dataspaces, filters, heaps, free-space metadata, and cache images	HDF5 is a complex binary metadata graph, not a flat byte container
application deserialization layered on HDF5	arbitrary code execution	Keras CVE-2025-9905: crafted `.h5/.hdf5` model archives can execute code because legacy `.h5` loading did not honor `safe_mode=True` for `Lambda`-layer pickled code [**cve-2025-9905**].	ML frameworks often use HDF5 as a model container, then deserialize framework-level objects from it.
external references and file path indirection	arbitrary local file read, path traversal, data exfiltration	Keras CVE-2026-1669: crafted `.keras` model file using HDF5 external dataset references could read local files [**cve-2026-1669**].	HDF5 supports links and external storage mechanisms. External links store both target filename and object path; external storage can resolve raw-data files through prefixes and current working directory rules [**hdf5-data-model**].
unbounded resource allocation	memory exhaustion / interpreter crash	Keras CVE-2026-0897: crafted `.keras` archive with `model.weights.h5` declaring an extremely large dataset shape could exhaust memory and crash Python [**cve-2026-0897**].	HDF5 metadata can declare logical shapes much larger than the actual useful payload; application code may allocate based on declared shape before policy checks.
dynamic plugin loading and search paths	local privilege escalation / arbitrary code loading	TensorFlow CVE-2026-2492: HDF5 plugin handling loaded plugins from an unsecured location, enabling local privilege escalation [**cve-2026-2492**].	HDF5 supports dynamic plugin loading, and plugin paths can be affected by defaults or `HDF5_PLUGIN_PATH`; dynamic loading can also be disabled via `H5PLset_loading_state()` or `HDF5_PLUGIN_PRELOAD=::` [**hdf5-dynamic-plugins**].

Table 18 – Common application-level HDF5 failure families, continued

Family	Typical failure	Examples	Why it is common
vulnerable or stale embedded HDF5 copies	inherited CVEs, unpatched parser behavior	matio CVE-2021-36977 involved a heap overflow related to use of HDF5 1.12.0; Apple CVE-2021-31009 says multiple issues were addressed by removing HDF5 [**cve-2021-36977**, **cve-2021-31009**].	Downstream products vendor, statically link, or lag HDF5 releases; vulnerability scanners then attribute HDF5 defects to the application.
helper-tool exposure	crashes or code execution through inspection/conversion utilities	`h5dump` appears in older and newer issues; CVE-2026-34734 describes a heap-use-after-free in the `h5dump` helper utility triggered by a malicious `.h5` file [**cve-2026-34734**].	Applications often call `h5dump`, `h5repack`, or converters as "safe inspection" tools on untrusted files.

The unifying root cause: The repeated mistake is *trust-boundary confusion*. HDF5 sits at the boundary between a file and the process. Downstream frameworks often add another boundary: model loading, dataset ingestion, scientific workflow execution, browser upload, cloud service processing, or plugin resolution. Many CVEs happen because those layers collapse into one assumption: "This is just an HDF5 file, so reading it is safe." That assumption is false. An HDF5 file can influence:

- memory allocation size
- dataset rank and shape
- chunk count and cache pressure
- filter/plugin loading
- external file lookup
- link traversal
- datatype conversion
- application-level model reconstruction
- tool output rendering

The common issue is **unsafe interpretation of HDF5-controlled metadata at the application boundary.**

6.1. Audit Findings and Mitigation Priorities

Mitigation has to be layered. Patching `libhdf5` alone will not fix Keras model deserialization, TensorFlow plugin path handling, or application-level external-reference policy.

Priority 1: Define an "untrusted HDF5" profile

HDF5 should offer, and downstreams should use, a strict profile for any file received from users, networks, archives, model hubs, object stores, email, web uploads, or third-party datasets.

Default behavior for that profile:

```
dynamic plugins disabled
external links disabled or sandboxed
external raw dataset storage disabled or sandboxed
VDS source traversal restricted
maximum rank enforced
maximum element count enforced
maximum allocation size enforced
maximum metadata bytes enforced
maximum chunk count enforced
maximum filter expansion ratio enforced
cycles and deep graph traversal rejected
unknown or legacy-dangerous messages rejected
    unless explicitly allowed
```

Today, pieces exist: plugin loading can be controlled, and all dynamic plugins can be disabled with `H5PLset_loading_state(0)` or `HDF5_PLUGIN_PRELOAD=::`. But this needs to become a cohesive security profile, not a set of hidden knobs.

Priority 2: Ship a preflight validator

Create a validator that reads metadata only and returns a policy decision before the application allocates large arrays or deserializes model objects.

Example output:

```
{
  "safe_for_untrusted_load": false,
  "reasons": [
    "external dataset reference outside sandbox",
    "dataset declares 8.7 TB logical size",
    "requires dynamic filter plugin",
    "contains legacy Keras Lambda layer metadata"
  ]
}
```

For core HDF5, this validator should reason about file-format invariants. For downstreams, it should expose application hooks:

```
on_external_reference(path)
on_dataset_shape(path, dtype, shape, logical_bytes)
on_filter_required(filter_id)
on_link_traversal(source, target)
on_attribute(name, size)
```

Keras and TensorFlow could then enforce model-specific policy before creating Python objects.

Priority 3: Disable dangerous HDF5 features by default in ML model loading

ML loaders should not behave like general HDF5 viewers.

For Keras/TensorFlow-style use:

```
do not allow Lambda or pickled code from legacy .h5
do not allow external dataset references
do not allow dynamic HDF5 plugins
do not eagerly allocate declared tensors before checking total
    byte budget
do not resolve paths relative to current working
    directory
do not honor HDF5_PLUGIN_PATH in model loading
    contexts
```

This directly addresses the Keras arbitrary-code issue, the Keras arbitrary-file-read issue, the Keras memory-exhaustion issue, and the TensorFlow plugin-search-path issue.

Priority 4: Make external references opt-in

External links and external raw dataset storage are legitimate HDF5 features, but they are unsafe defaults for hostile content. HDF5 documentation states that external links store a target filename and object path, and external dataset storage search behavior can involve the current working directory, absolute paths, relative paths, `${ORIGIN}`, and `HDF5_EXTFILE_PREFIX`.

Mitigation:

```
reject absolute external paths
reject path traversal
reject current-working-directory lookup
require explicit sandbox root
require explicit allowlist by extension or path
log every external resolution
never follow external references during metadata
    preview
```

For services, the safest default is: `externalreferences=off`.

Priority 5: Treat plugins as executable code

HDF5 filter plugins, VOL connectors, and VFDs are not data. They are code. The HDF5 docs state that plugins are shared libraries and that the default path can be overwritten by `HDF5_PLUGIN_PATH`. That is enough to create an application-level vulnerability if a privileged process uses an attacker-controlled path.

Mitigation:

```
clear HDF5_PLUGIN_PATH, HDF5_PLUGIN_PRELOAD,
    HDF5_EXTFILE_PREFIX in services
set plugin loading state explicitly at process start
allow plugins only from root/admin-owned directories
reject writable plugin directories
require hashes or signatures for plugins
never load plugins in setuid, service, notebook-hub,
    web-worker, or privileged contexts unless explicitly
    allowlisted
```

Sandboxing plugins…

Priority 6: Require downstream SBOM and version reporting

Downstream projects should report:

```
HDF5 version
linkage type: static / dynamic / vendored / system
enabled filters
enabled VOL/VFD connectors
plugin search path policy
external-reference policy
safe-mode policy
```

This solves a practical problem: a user seeing "HDF5 CVE" in a scanner often cannot tell whether they are affected through `libhdf5`, an official tool, a vendored copy, a Python wheel, a model loader, or a plugin.

Priority 7: Split CVE classification into scope tags

For HDF5-related CVE triage, use explicit tags:

```
HDF5-core-parser
HDF5-official-tool
HDF5-legacy-tool
HDF5-plugin-loading
HDF5-external-reference
HDF5-downstream-deserialization
HDF5-downstream-resource-policy
HDF5-vendored-library
not-HDF5
duplicate/rejected
```

This avoids bad prioritization. A heap overflow in `H5T__conv_struct_opt` is not the same class as Keras ignoring `safe_mode` for legacy `.h5` files, even though both mention HDF5.

A practical mitigation matrix for downstream maintainers

Table 19 – Minimum safe policies for downstream HDF5 use cases

Downstream use case	Minimum safe policy
viewing unknown `.h5` files	run in a sandbox; disable plugins; disable external references; cap metadata and output size
loading ML models	never load untrusted legacy `.h5`; disable Lambda/pickled code; validate tensor shapes before allocation

Table 19 – Minimum safe policies for downstream HDF5 use cases, continued

Downstream use case	Minimum safe policy
cloud upload service	preflight only; no plugin loading; no external file access; resource limits; process isolation
HPC workflow	separate trusted internal files from imported files; scrub environment; pin plugin directories; use module-level HDF5 version floors
desktop scientific app	prompt before following external references; ship patched HDF5; avoid writable global plugin dirs
package/distribution	publish SBOM; expose HDF5 version; rebuild promptly after HDF5 security releases
library binding	expose security controls in the binding API, not just through environment variables

What The HDF Group can do to reduce downstream CVEs: The most useful move is not just more parser hardening. It is to give downstreams safe controls that are easy to use.

Recommended deliverables:

```
H5Pset_security_profile(fapl, H5_SECURITY_PROFILE_UNTRUSTED)
H5Pset_external_reference_policy(fapl, H5_EXTREF_DENY
    | H5_EXTREF_SANDBOX)
H5Pset_plugin_policy(fapl, H5_PLUGIN_DENY
    | H5_PLUGIN_ALLOWLIST)
H5Pset_max_logical_dataset_bytes(fapl, n)
H5Pset_max_metadata_bytes(fapl, n)
H5Pset_max_object_count(fapl, n)
H5Pset_max_link_traversal_depth(fapl, n)
h5validate --profile untrusted --json file.h5
```

Then publish downstream recipes for Keras, TensorFlow, h5py, netCDF, MATLAB, R, Java, and cloud services.

7. HDF5 in the Software Supply Chain

- HDF5 2.0.0 source-artifact integrity check [**prisupp-2204**]
- SBOM

7.1. Upstream Dependencies

This subsection summarizes external software that can be required by an HDF5 build. It starts with the default serial C library and is organized by build option, because the default serial C library has a much smaller dependency set than feature-complete, testing, packaging, or developer builds. More details are provided in Appendix B.

HDF5's CMake build can either locate dependencies that are already installed or, for selected filter-related projects, add external source trees to the build with `FetchContent`. See table 22 for details.

The baseline HDF5 build has no mandatory external dependencies. The core library and C API tests can be built and run without any additional software. See table 23 for details.

The HDF5 build system has options that trigger additional dependencies when enabled. For example, enabling parallel HDF5 requires an MPI implementation, while enabling Fortran support requires a Fortran compiler. See table 24 for details.

Enabling filter plugins or the plugin project can introduce additional filter-library dependencies. The HDF5 cache defaults include entries for bitgroom, bitround, bitshuffle, Blosc, Blosc2, bzip2, fpzip, JPEG, LZ4, LZF, SZ, ZFP, and Zstandard. Those libraries are controlled by the plugin project's own options when `hdf5_plugins` is built or found. See table 25 for details.

Enabling VFDs for ROS3, HDFS, or other storage systems can introduce dependencies on external libraries and credentials. The same applies to VOL connectors. These are not filter dependencies, but they can affect the security and robustness of HDF5 file access, especially for third-party data. See table 26 for details.

Additional dependencies are required for optional tools, tests, and security features. For example, the `hdf5_security` target depends on OpenSSL when built with `HDF5_ENABLE_SECURITY_FEATURES`. See table 27 for details.

Finally, developer builds and packaging scripts have their own dependencies. These include code linters, formatters, static analyzers, documentation tools, and packaging tools. These dependencies are not required for a default HDF5 build, but they are relevant for the supply-chain risk of development and release processes. For example, if a security vulnerability is introduced in a packager or source formatter, it could affect the integrity of HDF5 releases even if the vulnerability is not present in the HDF5 source code itself. See table 28 for details.

Several default upstream source locations are encoded in `config/CacheURLs.cmake` for dependencies that HDF5 can fetch or whose values are passed to external filter-plugin builds. See table 29 for details.

7.2. Downstream Dependencies

For this audit, downstream dependencies are projects that compile against, dynamically load, package, or define data conventions on top of HDF5. The HDF5 ecosystem is much larger than any static list, so the most important downstreams are best identified by blast radius: broad installation footprint, long-lived archival data, domain-standard file conventions, or use as an intermediate layer for other tools. The following categories of downstreams are especially relevant.

Scientific data models and exchange formats.
The highest-impact downstream is netCDF-4, which uses HDF5 as the storage layer for the enhanced netCDF data model and is central to climate, weather, ocean, atmospheric, and other Earth-system data workflows [**netcdf4-format**]. NASA HDF-EOS5, NeXus, and CGNS similarly define domain-specific conventions over HDF5 for Earth observation, neutron/x-ray/muon facilities, and CFD or aerospace simulation data [**hdf-eos5**, **nexus-format**, **cgns-hdf5**]. For these formats, HDF5 compatibility is not just a library question: it affects

the readability of long-lived archives and the ability of independent tools to interpret a shared schema.

Language bindings and analysis libraries.
h5py, PyTables, pandas HDFStore, Bioconductor's rhdf5, HDF5.jl, and MATLAB's HDF5 interfaces make HDF5 a routine dependency for Python, R, Julia, and MATLAB users [**h5py-docs**, **pytables-docs**, **pandas-hdfstore**, **rhdf5**, **hdf5-jl**, **matlab-hdf5**]. These packages are important because they often sit at the boundary between upstream HDF5 builds and end-user scripts. Their packaging choices determine whether users load a bundled HDF5, a system HDF5, or an environment-provided HDF5, and therefore whether ABI, plugin-path, thread-safety, MPI, and filter support match the files being opened.

Geospatial, visualization, and inspection tools.
GDAL's HDF5 driver, ParaView/VTK-family builds, ITK's HDF5 module, HDFView, and similar readers expose HDF5 files to interactive and automated analysis workflows [**gdal-hdf5**, **paraview-build**, **itk-hdf5**]. This is an important security boundary: these tools are commonly used to inspect third-party data files, so robustness against malformed HDF5 objects, unusual filters, and unexpected VFD/plugin configuration has direct operational value.

HPC and parallel-I/O frameworks.
ADIOS2 includes an HDF5 engine, while netCDF-4, CGNS, h5py, and many simulation applications can be built against serial or parallel HDF5 variants [**adios2-hdf5**]. These downstreams amplify configuration risk: MPI ABI compatibility, collective I/O semantics, file locking behavior, subfiling support, and optional VFDs must line up across the application, HDF5, MPI, and filesystem stack.

Machine-learning and model-artifact workflows.
TensorFlow/Keras still supports older HDF5 model and weight files even as newer native formats are preferred [**tensorflow-keras-hdf5**]. This keeps HDF5 in the model-exchange and training-checkpoint ecosystem, usually through h5py, and makes HDF5 parser behavior relevant for model files obtained from external sources.

Table 20 – Representative downstream projects that depend on HDF5.

Downstream	Role	Supply-chain relevance
netCDF-4	Common scientific data model and format layered on HDF5.	Large transitive footprint through climate, weather, ocean, remote-sensing, and HPC data stacks. Uses a constrained HDF5 subset, so compatibility depends on both HDF5 behavior and netCDF conventions.
HDF-EOS5	NASA Earth-observation format built on HDF5 concepts and libraries.	Long-lived public archives and many third-party readers make backward compatibility and robust metadata handling high-value.
NeXus	Facility data convention for neutron, x-ray, and muon science.	Uses HDF5 groups, datasets, and attributes as a standardized experimental-data container, so HDF5 schema interpretation affects reproducibility across facilities.
CGNS	CFD and aerospace data standard with an HDF5 mapping.	Important for mesh and simulation-result exchange; parallel-HDF5 behavior and large-file support affect production simulation workflows.
h5py, PyTables, and pandas HDFStore	Python interfaces and tabular/array storage layers.	Common bridge between binary HDF5 files and Python analysis. Wheel and conda packaging can hide which HDF5 build is actually loaded.
rhdf5, HDF5.jl, MATLAB HDF5 APIs	R, Julia, and MATLAB access layers.	Expose HDF5 to laboratory and scientific-computing workflows where files are often exchanged outside controlled build environments.

GDAL and geospatial readers	Raster and subdataset access for HDF5-based products.	Converts HDF5 issues into geospatial ingestion issues, especially for remote-sensing products with complex metadata and nested subdatasets.
ParaView/VTK, ITK, HDFView, and viewers	Interactive inspection, visualization, and conversion.	Frequently open untrusted or externally generated files; defensive parsing, filter availability, and plugin search paths matter.
ADIOS2 and simulation I/O layers	Alternative I/O stacks with HDF5 read/write engines or compatibility paths.	Parallel and high-throughput workflows depend on matching MPI, filesystem, and HDF5 feature assumptions across the whole stack.
TensorFlow/Keras HDF5 artifacts	Legacy model and weight serialization.	Keeps HDF5 in ML artifact exchange; malformed or incompatible model files surface through h5py and HDF5 library behavior.

7.3. Audit Findings and Mitigation Priorities

- HDF5 changes have a large transitive blast radius because many users reach HDF5 through netCDF, GDAL, h5py, MATLAB, or domain formats rather than through the HDF5 C API directly.
- ABI and packaging mismatches are a recurring downstream risk. Python, R, Julia, MATLAB, MPI, and system-package environments may all load different HDF5 builds in the same organization.
- File parsing is a practical security boundary. Viewers, converters, notebooks, and data-ingestion jobs often process files received from outside the organization that built the HDF5 library.

- Optional features can become hidden interoperability requirements: filter plugins, SZIP/zlib variants, ROS3/HDFS/VFD support, parallel HDF5, and file-locking behavior may be assumed by downstream data even when they are not present in a default HDF5 build.
- Long-lived scientific archives make deprecation slower than ordinary application software. Even when a downstream project prefers a newer format, historic HDF5 files remain operationally relevant.

8. Independent HDF5 Implementations

8.1. Overview

- HDF5 library playing footsie with the file format spec: GitHub issue 6409 [**hdf5-gh-issue-6409**]

PureHDF Managed .NET read/write without native binaries

jHDF JVM HDF5 reading

netCDF-Java JVM netCDF/HDF-EOS/HDF5 scientific-data reading

pyfive Pure Python read-only inspection

jsfive Browser-side direct HDF5 read

scigolib/hdf5 Pure Go read/write

hdf5-pure and rustyhdf5-format/rustyhdf5 Pure Rust read/write

async-hdf5 or h5coro Cloud object-store chunk mapping

JLD2.jl Julia-native persistence in HDF5-compatible files

8.2. Audit Findings and Mitigation Priorities

The main risk across all direct-format implementations is not opening simple files; it is coverage of the long tail: dense groups, fractal heaps, shared object-header messages, all chunk-index variants, references, VLEN, compound/nested datatypes, virtual datasets, external storage, SWMR flags, user blocks, non-core filters, unusual libver bounds, and files produced by different HDF5 library releases. For any production use, build a compatibility corpus and round-trip the generated files using `h5dump`, `h5debug`, `h5py/libhdf5`, and the target-independent implementation.

Machine-readable specification: `hdf5-pickles`

9. Long-term Data Accessibility and Interpretability

Long term data accessibility and interpretability [**forum-plugin**]

9.1. Audit Findings and Mitigation Priorities

10. Resources

- HDF SSP SIG [**hdf5-ssp-sig**]
- Tools
  - HDF5 CVE test suite [**cve_hdf5**]
  - A machine-readable specification of the HDF5 file format. [**hdf5-pickles**]
  - HDF5 extension templates [**hdf5-boneyard**]
  - HDF5 plugins [**hdf5_plugins**]
- S2D2

A. CVE History Summary

Table 21 – HDF5 CVE history summary

CVE(s)	Affected area	Reported failure mode	Root cause summary
CVE-2025-7069, CVE-2025-6270, CVE-2025-2914	file-space info / free-space manager	heap buffer overflow	file-derived file-space metadata, especially page size / section data, was accepted without sufficient sanity checks. Fix rejects invalid page size and corrupted free-space info early.
CVE-2025-6858, CVE-2025-6817, CVE-2025-2913, CVE-2025-2912	object-header / metadata-cache cascade	null dereference, resource consumption, use-after-free, heap overflow	corrupted object-header continuation metadata caused later cache/free-list/object-message failures. Root cause is bad continuation-message size/state not rejected at decode time.
CVE-2025-6856, CVE-2025-6816, CVE-2025-2923	object-header continuation messages	heap buffer overflow / use-after-free	crafted object header with continuation message pointing back to itself led to under-sized internal buffer allocation. Fix compares expected and actual object-header chunk counts during deserialization.
CVE-2025-6750, CVE-2024-33874	mtime object-header messages	heap buffer overflow	old/new modification-time messages were not properly decoded; an invalid message size could produce a zero-size buffer passed to an encoder. Fix decodes both mtime formats and rejects invalid size.
CVE-2025-6269, CVE-2025-6516	metadata cache image / address decode	heap buffer overflow	cache-image reconstruction used insufficient bounds and input validation. Fix adds buffer-size tracking, overflow checks, input validation, and cleanup on failure.
CVE-2025-44905, CVE-2025-2308, CVE-2024-29159, CVE-2024-32615, CVE-2016-4331	scale-offset / n-bit filters	heap overflow / buffer overrun / arbitrary code execution	filter decoders trusted encoded precision, byte counts, or scale-offset parameters from the file. Root cause is insufficient bounds checking in decompression/filter parameter handling.
CVE-2025-44904, CVE-2024-32614, CVE-2024-32605, CVE-2024-29165, CVE-2019-9151, CVE-2018-14035, CVE-2018-13875, CVE-2018-11205	vectorized memory copy / scatter-fill paths	heap/stack overflow or overread	memory-vector copy routines copied between mismatched source/destination selections or trusted element counts too far. Root cause is insufficient validation of vector extents and copy sizes before `memcpy`-style operations.
CVE-2025-2926	object-header cache checksum serialization	null pointer dereference	object-header checksum/serialization path could operate on incomplete or invalid object-header state. Root cause is missing null/state validation before serialization.
CVE-2025-2925	memory management wrapper	double free	reallocation/error path mishandled ownership of `mem`. Root cause is ambiguous ownership/cleanup semantics around failed or edge-case realloc.
CVE-2025-2924, CVE-2024-32613, CVE-2024-32612	local heap free-list deserialization	heap buffer overflow	local-heap free-block metadata from the file was trusted. Root cause is insufficient validation of free-block size/offset before rebuilding heap structures.

Table 21 – HDF5 CVE history summary, continued

CVE(s)	Affected area	Reported failure mode	Root cause summary
CVE-2025-2915, CVE-2018-13867	file accumulator	heap overflow / out-of-bounds read	accumulator overlap/offset math accepted invalid ranges. Root cause is weak range validation for file accumulator reads/frees.
CVE-2025-2310, CVE-2019-9152	datatype / attribute string decoding	heap buffer overflow / out-of-bounds read	string duplication from metadata used file-derived length or unterminated data. Root cause is unsafe string length handling during datatype/attribute decode.
CVE-2025-2309, CVE-2024-29163	datatype bit operations	heap buffer overflow	bit-copy / bit-find logic trusted bit offsets, lengths, or precision metadata. Root cause is insufficient bounds checking in datatype bitfield operations.
CVE-2025-2153	shared-message deletion	heap buffer overflow	shared-message deletion path could process corrupted shared-message/object-header state. Root cause appears to be insufficient validation of shared-message metadata and cleanup state before deletion.
CVE-2025-26200, CVE-2024-33877, CVE-2017-17507	compound datatype conversion	heap overflow / out-of-bounds read/write	compound datatype conversion trusted complex datatype layout details. Root cause is unsafe datatype conversion when member layout, unused bits, or conversion metadata are malformed.
CVE-2024-33876	point selection deserialization	heap overflow	serialized point-selection metadata was trusted. Root cause is missing bounds/count validation when rebuilding point selections.
CVE-2024-33875, CVE-2019-8396	dataset layout encode	heap/buffer overflow	dataset layout metadata could encode inconsistent dimensions/storage sizes. Fix family adds dataset layout size checks, multiplication overflow checks, and file-bound checks.
CVE-2024-33873	dataset scatter memory	heap overflow	scatter operation accepted inconsistent memory/file selection sizes. Root cause is insufficient validation of selected element counts and destination capacity.
CVE-2024-32624, CVE-2024-32610	reference datatype memory nulling	heap overflow	reference-memory initialization/nulling used malformed datatype/reference metadata. Root cause is missing size/type validation before clearing reference memory.
CVE-2024-32623	generic array fill	heap overflow	array fill operation trusted element count/size metadata. Root cause is missing multiplication/range validation before filling memory.
CVE-2024-32622, CVE-2024-29158, CVE-2024-29161 note	free-list array allocation	out-of-bounds read / stack overflow	array allocation/free-list path accepted invalid counts or sizes. Root cause is weak validation of allocation quantity derived from file metadata.
CVE-2024-32621, CVE-2024-29162, CVE-2024-29157	global heap read	heap/stack overflow	global heap object size/offset metadata could be inconsistent with actual buffer. Root cause is insufficient validation of heap object boundaries before read.
CVE-2024-32620, CVE-2025-6516, CVE-2021-45830, CVE-2018-13866	address-length decode	heap/stack overflow / over-read	encoded file addresses/lengths could exceed expected bounds. Root cause is insufficient validation of decoded address/length byte counts and destination capacity.

Table 21 – HDF5 CVE history summary, continued

CVE(s)	Affected area	Reported failure mode	Root cause summary
CVE-2024-32619, CVE-2018-14031	datatype copy/reopen	heap overflow / over-read	datatype copy path accepted malformed nested datatype metadata. Root cause is missing validation while copying or reopening datatype structures.
CVE-2024-32618	native datatype conversion	heap overflow	native-type construction trusted malformed datatype metadata. Root cause is inadequate validation during recursive datatype normalization.
CVE-2024-32617	`H5MM_xstrdup`	heap overflow	unsafe string duplication from file-derived metadata. Root cause is missing bounded string handling.
CVE-2024-32616	datatype object-header encode helper	heap overflow	datatype message encode helper accepted inconsistent datatype-message size/state. Root cause is missing encode-time size validation.
CVE-2024-32611	attribute release table	uninitialized value / DoS	attribute-table bookkeeping conflated allocated capacity and actual attribute count. Fix separates current count and maximum capacity and cleans partial tables on error.
CVE-2024-32608, CVE-2024-32607	attribute close	memory corruption / segfault	attribute close/release path could operate on partially initialized or corrupted attribute state. Root cause is unsafe cleanup after failed attribute-table construction or decode.
CVE-2024-32606	h5tools string printing	uninitialized dereference / DoS	tool-side string formatting could dereference uninitialized values. No commit URL listed, so fix review was not possible.
CVE-2024-29166	link-info decode	buffer overrun	object link-info metadata decode trusted malformed encoded size/flags. No commit URL listed.
CVE-2022-26061, CVE-2022-25972, CVE-2022-25942, CVE-2020-10809, CVE-2018-17436, CVE-2018-17433	`gif2h5` tool / GIF parser	heap overflow, out-of-bounds write/read, invalid write	legacy GIF conversion code parsed attacker-controlled GIF/HDF inputs unsafely. Commentary says the tool was removed rather than surgically hardened.
CVE-2021-46244	datatype copy completion	divide by zero / DoS	datatype copy logic failed to guard arithmetic denominators or zero-sized datatype state.
CVE-2021-46243	datatype object-header decode	untrusted pointer dereference / DoS	datatype decode accepted malformed pointer/metadata state. Root cause is missing validation before dereference.
CVE-2021-46242, CVE-2020-10810	metadata cache pin/unpin	use-after-free / null dereference	cache entry lifecycle operations did not robustly handle malformed or unexpected cache state.
CVE-2021-45833	chunk file-map hyperslab	stack overflow / DoS	chunk/hyperslab mapping trusted malformed dataspace/chunk metadata. Root cause is inadequate bounds checking in chunk mapping.
CVE-2021-45832	error stack internals	stack overflow / DoS	listed as cannot reproduce; no fix commit. Treat root cause as unconfirmed.
CVE-2021-45829	unspecified HDF5 path	segfault / DoS	Root cause undetermined.
CVE-2021-37501	`h5dump` string sprint	buffer overflow / DoS	h5dump formatting path wrote beyond buffer while rendering file-derived data.
CVE-2021-36977	mat file I/O library using HDF5	heap overflow	not filed against HDF5. Root cause belongs to downstream integration/context, not the HDF5 fix set.

Table 21 – HDF5 CVE history summary, continued

CVE(s)	Affected area	Reported failure mode	Root cause summary
CVE-2021-31009	Apple platform issue	multiple HDF5 issues	not filed against HDF5; fixed by Apple by removing HDF5 from affected components.
CVE-2020-18494, CVE-2020-18232	dataspace close	buffer overflow / possible RCE	malformed file could corrupt dataspace cleanup/close state. Root cause is unsafe lifecycle handling of malformed dataspace metadata.
CVE-2020-10812	file reference count query	null pointer dereference / DoS	file query path failed to validate file/reference state before dereference.
CVE-2020-10811, CVE-2018-14033	layout decode	heap over-read	dataset layout message decoder trusted encoded layout fields. Root cause is missing bounds checks around decoded layout data.
CVE-2019-8398, CVE-2019-8397	datatype size/close	out-of-bounds read	datatype object state could be malformed before size/close operations. No commit URL listed.
CVE-2018-17439	dataspace extent dimensions during HDF to GIF conversion	stack overflow	conversion tool path trusted dataspace dimensions from file.
CVE-2018-17438	selected I/O	divide by zero / DoS	selected-I/O path failed to guard divisor/count metadata from crafted file. Tool-removal commit, so the listed fix is coarse.
CVE-2018-17437	datatype decode helper	memory leak / DoS	error path leaked allocations when datatype metadata was malformed.
CVE-2018-17435	attribute decode	heap over-read	attribute message decoder trusted encoded sizes/offsets.
CVE-2018-17434	h5repack filter application	divide by zero / DoS	tool-side filter setup failed to guard zero-valued filter/chunk parameters.
CVE-2018-17432	simple dataspace encode	null pointer dereference / DoS	dataspace encode path accepted invalid dataspace state before dereference.
CVE-2018-17237, CVE-2018-17233, CVE-2018-11207, CVE-2018-11203, CVE-2017-17508	chunking / b-tree / datatype arithmetic	divide by zero / SIGFPE / DoS	crafted file could set chunk, b-tree, datatype, or layout parameters to zero. Root cause is missing arithmetic precondition checks.
CVE-2018-17234, CVE-2018-13873, CVE-2018-11204	object-header chunk deserialize	memory leak / over-read / null dereference	object-header chunk deserialization accepted malformed chunk metadata and mishandled error cleanup.
CVE-2018-16438	external links	out-of-bounds read	external-link query trusted malformed external-link metadata.
CVE-2018-15672, CVE-2018-14032	duplicate/rejected IDs	not applicable	rejected duplicate CVEs; no HDF5 root cause to summarize beyond the referenced canonical CVEs.
CVE-2018-15671	property-list callback lookup	excessive stack consumption / DoS	recursive or malformed property-list callback state allowed stack exhaustion.
CVE-2018-14460	simple dataspace decode	heap over-read	dataspace message decoder trusted encoded extent/rank metadata.
CVE-2018-14034	filter pipeline reset	out-of-bounds read	malformed filter-pipeline metadata caused unsafe reset/read behavior. No commit URL listed.

Table 21 – HDF5 CVE history summary, continued

CVE(s)	Affected area	Reported failure mode	Root cause summary
CVE-2018-13876, CVE-2018-13874	sec2 VFD read path	stack overflow	low-level file-driver read/memset path could be driven out of bounds by malformed file metadata. No commit URL listed.
CVE-2018-13872	group entry decode	heap overflow	group-entry decoder trusted malformed symbol-table/group metadata. No commit URL listed.
CVE-2018-13871	free-list block allocation	heap overflow	free-list block allocator accepted invalid allocation size/count.
CVE-2018-13870, CVE-2018-13869	link decode	heap over-read / overlapping memcpy	link-message decoder trusted malformed link metadata, including overlapping source/destination regions.
CVE-2018-13868	old fill-value decode	heap over-read	old fill-value decoder trusted encoded fill-value size. No commit URL listed.
CVE-2018-11206	fill-value decode	out-of-bounds read / possible disclosure	fill-value message decoders trusted encoded size/type fields.
CVE-2018-11202	hyperslab spans	null pointer dereference / DoS	hyperslab span construction failed to validate malformed selection state.
CVE-2017-17509	group cache entry decode vector	out-of-bounds write	group-entry vector decoder trusted file-derived counts/array bounds. No commit URL listed.
CVE-2017-17506, CVE-2017-17505	filter pipeline decode	out-of-bounds read / null dereference	pipeline decoder failed to validate malformed filter counts or fields. No commit URL listed.
CVE-2016-4333	array allocation / initialization loop	out-of-bounds write	file-controlled value could modify loop termination while initializing an allocated array.
CVE-2016-4332	object-header message flags	heap out-of-bounds write / code execution	library failed to check whether a message type supported a flag before casting to an incompatible structure.
CVE-2016-4330	dataspace array rank/dimensions	heap overflow / code execution	file-supplied array dimensionality was not checked against allocated space.

B. Software Supply Chain Details

The HDF5_ALLOW_EXTERNAL_SUPPORT modes apply to filter-related dependencies. Some test and example configurations have their own source-fetch paths, such as KWSys for the API test driver and h5py for Python examples. MPI, HDFS, ROS3, OpenSSL, Java, documentation, packaging, and developer tools must be provided by the build environment.

Table 22 – Dependency source modes

Mode	Relevant options	Meaning
System packages	HDF5_ALLOW_EXTERNAL_SUPPORT=NO	CMake uses find_package(), find_library(), or find_program() to locate installed dependencies. This is the default.
Git fetch	HDF5_ALLOW_EXTERNAL_SUPPORT=GIT plus a dependency-specific *_USE_EXTERNAL=ON option	CMake downloads source from the configured *_GIT_URL and *_GIT_TAG values and builds it with HDF5.
Tarball fetch	HDF5_ALLOW_EXTERNAL_SUPPORT=TGZ plus a dependency-specific *_USE_EXTERNAL=ON option	CMake downloads from *_TGZ_ORIGPATH and *_TGZ_NAME, or uses TGZPATH when the dependency-specific *_USE_LOCALCONTENT=ON option is set.

Table 23 – Baseline build

Dependency	Trigger	Notes
CMake 3.26 or newer	Always	Required by the top-level cmake_minimum_required().
C compiler and native build tool	Always	The top-level project is a C project.
Platform C library, headers, and system calls	Always	CMake probes standard headers, types, functions, byte order, large-file support, file locking, pread/pwrite, and related platform features.
Platform link libraries	Platform-dependent	CMake adds libraries only when probes show they are needed: m, dl, rt, posix4, ucb, ws2_32, wsock32, and shlwapi.

PkgConfig	Optional probe	Used only when available. It can help some find modules, such as zlib-ng and libaec, but it is not a hard requirement for the default build.
Threads	Found in every configure; required by thread features	CMake searches for C11 threads, Win32 threads, or pthreads. When found, HDF5 records thread support and links `Threads::Threads`; `HDF5_ENABLE_THREADSAFE`, `HDF5_ENABLE_CONCURRENCY`, and `HDF5_ENABLE_SUBFILING_VFD` require it.

Table 24 – HDF5 feature triggers

Feature or component	Build options	Dependency	Notes
Parallel HDF5	`HDF5_ENABLE_PARALLEL=ON`	MPI C with MPI-IO, MPI standard 3.0 or newer	Adds `MPI::MPI_C` to public link libraries. Parallel filtered writes and large parallel I/O are enabled only when the required MPI symbols are present. Parallel tests also need an MPI launcher such as `mpiexec`.
Parallel Fortran	`HDF5_ENABLE_PARALLEL=ON`, `HDF5_BUILD_FORTRAN=ON`	MPI Fortran	The Fortran build adds MPI Fortran libraries. The `mpi_f08` module is used when available.
Thread-safe library	`HDF5_ENABLE_THREADSAFE=ON`	Threads	Unsupported with parallel, Fortran, C++, or high-level library unless `HDF5_ALLOW_UNSUPPORTED=ON`. On Windows, static libraries are rejected.

Multi-threaded concurrency	HDF5_ENABLE_CONCURRENCY=ON	Threads	Experimental. Mutually exclusive with HDF5_ENABLE_THREADSAFE; has the same unsupported combinations as thread-safety unless overridden with HDF5_ALLOW_UNSUPPORTED=ON.
C++ wrappers	HDF5_BUILD_CPP_LIB=ON	C++ compiler	Unsupported with parallel unless HDF5_ALLOW_UNSUPPORTED=ON. High-level C++ wrappers also require HDF5_BUILD_HL_LIB=ON.
Fortran wrappers	HDF5_BUILD_FORTRAN=ON	Fortran compiler with Fortran 2003 ISO_C_BINDING support	Unsupported with thread-safety or concurrency unless HDF5_ALLOW_UNSUPPORTED=ON. High-level Fortran wrappers also require HDF5_BUILD_HL_LIB=ON.
Java wrappers	HDF5_BUILD_JAVA=ON	Java/JDK, shared HDF5 libraries, Java dependency JARs	Java requires shared HDF5 libraries. Java 11 or newer is required for the JNI implementation. Java tests additionally require the JUnit and Hamcrest JARs.
Java JNI implementation	HDF5_BUILD_JAVA=ON, HDF5_ENABLE_JNI=ON, or any Java version earlier than 25	JNI headers and libraries	JNI is the default Java implementation. HDFS also requires JNI.
Java FFM implementation	HDF5_BUILD_JAVA=ON, Java 25 or newer, HDF5_ENABLE_JNI=OFF	jextract from JEXTRACT_HOME/bin or JAVA_HOME/bin	Generates FFM bindings at configure time.

Java logging/test JARs	`HDF5_BUILD_JAVA=ON`	`slf4j-api`, `slf4j-nop`, `slf4j-simple`; plus JUnit 4 and Hamcrest for tests	The default paths point to bundled JARs under `java/lib`. They can be overridden with `HDF5_JAVA_*_JAR` cache variables. `HDF5_ENABLE_MAVEN_DEPLOY=ON` generates Maven-style artifacts and POM files; the CMake build does not locate or run `mvn`.
Documentation	`HDF5_BUILD_DOC=ON`	Doxygen	`HDF5_ENABLE_DOXY_WARNINGS=ON` makes Doxygen warnings fail the build.
Header regeneration	`HDF5_GENERATE_HEADERS=ON`	Perl	Perl is optional otherwise. If it is missing, generated headers are not regenerated.
Python examples	`H5EXAMPLE_BUILD_PYTHON=ON`	Python3 interpreter, Python development files, NumPy, h5py source, pip	The examples CMake project fetches `h5py` from Git and installs it with `python3 -m pip`.

Table 25 – HDF5 filter dependency choices

Feature	Build options	Dependency	Source choices
DEFLATE filter	`HDF5_ENABLE_ZLIB_SUPPORT=ON`, `HDF5_USE_ZLIB_NG=OFF`	zlib	System package by default; `ZLIB_USE_EXTERNAL=ON` can build zlib from Git or TGZ. `HDF5_USE_ZLIB_STATIC=ON` prefers a static library.

DEFLATE filter through zlib-ng	HDF5_ENABLE_ZLIB_SUPPORT=ON, HDF5_USE_ZLIB_NG=ON	zlib-ng	System package by default; ZLIB_USE_EXTERNAL=ON can build zlib-ng from Git or TGZ. zlib compatibility mode is detected.
SZIP filter	HDF5_ENABLE_SZIP_SUPPORT=ON	libaec with libsz compatibility, or another package selected by LIBAEC_PACKAGE_NAME	System package by default; SZIP_USE_EXTERNAL=ON can build libaec from Git or TGZ. HDF5_USE_LIBAEC_STATIC=ON prefers static libraries.
HDF5 filter plugins project	HDF5_ENABLE_PLUGIN_SUPPORT=ON	Installed h5pl/HDF5 filter plugins package, or the hdf5_plugins project	PLUGIN_USE_EXTERNAL=ON can build the plugin project from Git or TGZ. Plugin support is disabled for the 1.6 API.

Table 26 – HDF5 VFD and VOL connector dependency choices

Feature	Build options	Dependency	Notes
Direct VFD	HDF5_ENABLE_DIRECT_VFD=ON	POSIX O_DIRECT and posix_memalign() support	No third-party library. Configuration fails if the platform support is missing.
Mirror VFD	HDF5_ENABLE_MIRROR_VFD=ON	POSIX socket/fork headers and functions	No third-party library. Requires netinet/in.h, netdb.h, arpa/inet.h, sys/socket.h, and fork().

ROS3 VFD	`HDF5_ENABLE_ROS3_VFD=ON`	`aws-c-s3` package	CMake	HDF5 does not fetch this library. Building `aws-c-s3` from source can also require AWS C libraries such as `aws-c-common`, `aws-checksums`, `aws-c-cal`, `aws-c-io`, `aws-c-compression`, `aws-c-http`, `aws-c-sdkutils`, `aws-c-auth`, and, on Linux, `aws-lc` and `s2n-tls`. ROS3 docker-proxy tests additionally require `docker` and the AWS CLI.
HDFS VFD	`HDF5_ENABLE_HDFS=ON`	JNI/JVM and Hadoop `libhdfs`		`HADOOP_HOME` is used to locate `hdfs.h`, `libhdfs`, and the Hadoop version command. Non-MSVC builds also add `-pthread`.
Subfiling VFD and IOC VFD	`HDF5_ENABLE_PARALLEL=ON`, `HDF5_ENABLE_SUBFILING_VFD=ON`	MPI C with `MPI_Comm_split_type`, plus Threads		Not available on Windows. IOC VFD is built when subfiling is enabled.
External VOL connectors	`HDF5_VOL_ALLOW_EXTERNAL=GIT` or `LOCAL_DIR` with `HDF5_VOL_URLnn` or `HDF5_VOL_PATHnn`	Connector-defined		Up to ten CMake-based VOL connectors can be added. HDF5 patches connector CMake code to use the in-tree HDF5 build, but each connector may bring its own dependencies.

Table 27 – Tools, Tests, and Security Features

Feature	Build options	Dependency	Notes
Signed plugin verification	`HDF5_REQUIRE_SIGNED_PLUGINS=ON`	OpenSSL libcrypto 1.1.0 or newer	Adds `OpenSSL::Crypto` to HDF5. `HDF5_LOCK_PLUGIN_KEYSTORE=ON` also requires `HDF5_PLUGIN_KEYSTORE_DIR`.
`h5sign` tool	`HDF5_REQUIRE_SIGNED_PLUGINS=ON`, `HDF5_BUILD_TOOLS=ON`	OpenSSL libcrypto	Used to sign HDF5 plugins.
Signed-plugin tests	`HDF5_REQUIRE_SIGNED_PLUGINS=ON`, `BUILD_TESTING=ON`	`openssl` executable	Tests generate RSA keys with the OpenSSL command-line tool.
Parallel tools utilities	`HDF5_ENABLE_PARALLEL=ON`, `HDF5_BUILD_PARALLEL_TOOLS=ON`	MFU, CIRCLE, DTCMP	CMake looks for headers and libraries, using `MFU_ROOT` as a hint.
API test driver	`BUILD_TESTING=ON`, `HDF5_TEST_API_ENABLE_DRIVER=ON`	KWSys	The test driver fetches KWSys from a tarball or from `TGZPATH` when `KWSYS_USE_LOCALCONTENT=ON`.
Shell-script tests	`BUILD_TESTING=ON`	PowerShell or Bash	CMake uses `pwsh`/`powershell` when available; otherwise Unix builds look for `bash`.
ROS3 VFD tests	`HDF5_ENABLE_ROS3_VFD=ON`, `HDF5_ENABLE_ROS3_VFD_DOCKER_PROXY=ON`	Docker daemon and AWS CLI	Used to run S3 proxy tests.

Table 28 – Developer and Packaging Dependencies

Configuration	Build options	Dependency
Analyzer tooling	`HDF5_ENABLE_ANALYZER_TOOLS=ON`	`clang-tidy`, `include-what-you-use`, and `cppcheck` are probed and used by the analyzer macros.
Source formatting targets	`HDF5_ENABLE_FORMATTERS=ON`	`clang-format` and `cmake-format`.
Code coverage with Clang/IntelLLVM	`HDF5_ENABLE_COVERAGE=ON`, `CODE_COVERAGE=ON`	`llvm-cov` and `llvm-profdata`; AppleClang uses `xcrun llvm-cov` and `xcrun llvm-profdata`.
Code coverage with GCC	`HDF5_ENABLE_COVERAGE=ON`, `CODE_COVERAGE=ON`	`lcov` and `genhtml`; fallback instrumentation can also link `gcov`.
Sanitizers	`HDF5_ENABLE_SANITIZERS=ON`, `HDF5_USE_SANITIZER=<kind>`	Compiler sanitizer runtime support. Supported values depend on compiler and platform.
AFL support module	Explicit inclusion of `config/sanitizer/afl-fuzzing.cmake`	`afl-cc` and `afl-c++`. This module is available but is not included by the top-level HDF5 build by default.
Dependency graph support module	Explicit inclusion of `config/sanitizer/dependency-graph.cmake`	Graphviz `dot`. This module is available but is not included by the top-level HDF5 build by default.
Windows installers	CPack on Windows	NSIS and WiX are probed when available.
Linux packages	CPack on Unix-like systems	`dpkg-shlibdeps` enables DEB generation; `rpmbuild` enables RPM generation for non-parallel builds.

Table 29 – External Source Defaults

Dependency	Default version/tag	Default source

zlib	1.3.2 / v1.3.2	https://github.com/madler/zlib.git or https://github.com/madler/zlib/releases/download/v1.3.2/zlib-1.3.2.tar.gz
zlib-ng	2.3.3	https://github.com/zlib-ng/zlib-ng.git or https://github.com/zlib-ng/zlib-ng/archive/refs/tags/2.3.3.tar.gz
libaec	1.1.6 / v1.1.6	https://github.com/MathisRosenhauer/libaec.git or https://github.com/MathisRosenhauer/libaec/releases/download/v1.1.6/libaec-1.1.6.tar.gz
HDF5 filter plugins	master	https://github.com/HDFGroup/hdf5_plugins.git or https://github.com/HDFGroup/hdf5_plugins/releases/download/snapshot/hdf5_plugins-master.tar.gz
KWSys API-test helper	master tarball	https://gitlab.kitware.com/utils/kwsys/-/archive/master/kwsys-master.tar.gz
h5py Python examples	master	https://github.com/h5py/h5py.git
bitshuffle	master or 0.5.2 tarball	https://github.com/kiyo-masui/bitshuffle.git or https://github.com/kiyo-masui/bitshuffle/archive/refs/tags/bitshuffle-0.5.2.tar.gz
Blosc	1.21.6	https://github.com/Blosc/c-blosc.git or https://github.com/Blosc/c-blosc/archive/refs/tags/c-blosc-1.21.6.tar.gz
Blosc zlib helper	develop or 1.3.1 tarball	https://github.com/madler/zlib.git or https://github.com/madler/zlib/releases/download/v1.3.1/zlib-1.3.1.tar.gz
Blosc2	2.17.1	https://github.com/Blosc/c-blosc2.git or https://github.com/Blosc/c-blosc2/archive/refs/tags/c-blosc2-2.17.1.tar.gz

Blosc2 zlib helper	`develop` or `1.3.1` tarball	https://github.com/madler/zlib.git or https://github.com/madler/zlib/releases/download/v1.3.1/zlib-1.3.1.tar.gz
bzip2	`bzip2-1.0.8`	https://github.com/libarchive/bzip2.git or https://github.com/libarchive/bzip2/archive/refs/tags/bzip2-bzip2-1.0.8.tar.gz
fpzip	`develop` or `1.3.0` tarball	https://github.com/LLNL/fpzip.git or https://github.com/LLNL/fpzip/releases/download/1.3.0/fpzip-1.3.0.tar.gz
JPEG	`jpeg-9e`	https://github.com/libjpeg-turbo/libjpeg-turbo.git or https://www.ijg.org/files/jpegsrc.v9e.tar.gz
LZ4	`dev` or `1.10.0` tarball	https://github.com/lz4/lz4.git or https://github.com/lz4/lz4/releases/download/v1.10.0/lz4-1.10.0.tar.gz
LZF	`3.6` tarball	http://dist.schmorp.de/liblzf/liblzf-3.6.tar.gz
SZ	`master` or `2.1.12.5` tarball	https://github.com/szcompressor/SZ.git or https://github.com/szcompressor/SZ/releases/download/v2.1.12.5/SZ-2.1.12.5.tar.gz
ZFP	`develop` or `1.0.1` tarball	https://github.com/LLNL/zfp.git or https://github.com/LLNL/zfp/releases/download/1.0.0/zfp-1.0.1.tar.gz
Zstandard	`1.5.7`	https://github.com/facebook/zstd.git or https://github.com/facebook/zstd/releases/download/v1.5.7/zstd-1.5.7.tar.gz